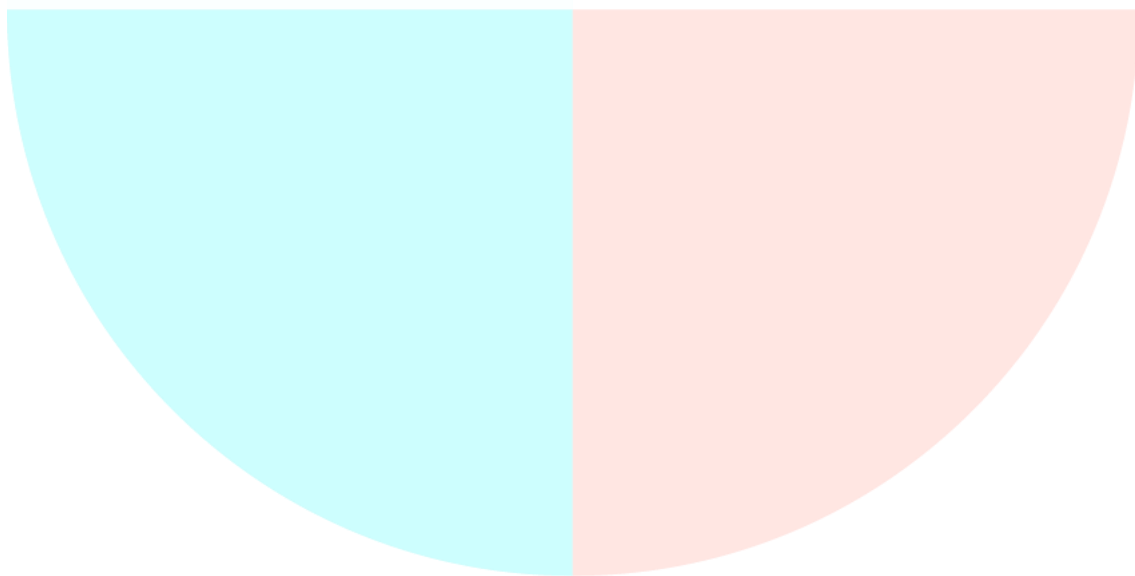


Bloque 2 - Tema 3

TIPOS ABSTRACTOS Y ESTRUCTURAS DE DATOS.
ORGANIZACIONES DE FICHEROS. ALGORITMOS.
FORMATOS DE INFORMACIÓN Y FICHEROS



PREPARACIÓN OPOSICIONES
TÉCNICOS AUXILIARES DE INFORMÁTICA

ÍNDICE

ÍNDICE	2
1. TIPOS ABSTRACTOS Y ESTRUCTURAS DE DATOS	3
1. <i>Arrays</i>	5
2. <i>Registros</i>	7
3. <i>Listas</i>	8
4. <i>Pilas</i>	11
5. <i>Colas</i>	13
6. <i>Conjuntos</i>	14
7. <i>Mapas</i>	15
8. <i>Diccionarios</i>	15
9. <i>Tabla hash</i>	15
10. <i>Árboles</i>	16
11. <i>Grafos</i>	25
2. ALGORITMOS	29
1. <i>Eficiencia de un algoritmo</i>	29
2. <i>Recursividad</i>	33
3. <i>Algoritmos de búsqueda</i>	35
4. <i>Algoritmos de ordenación o clasificación</i>	37
5. <i>Algoritmos teoría de grafos</i>	42
3. ORGANIZACIONES DE FICHEROS	43
1. <i>Organización secuencial</i>	43
2. <i>Organización directa</i>	44
3. <i>Variantes de la organización secuencial</i>	45
4. FORMATOS DE INFORMACIÓN Y FICHEROS	50
1. <i>Formatos de imágenes</i>	50
2. <i>Formatos de documentos</i>	53
3. <i>Formatos de intercambio de datos</i>	53
4. <i>Formatos de audio y video</i>	57

1. TIPOS ABSTRACTOS Y ESTRUCTURAS DE DATOS

Para empezar, hay que precisar la diferencia entre tres conceptos muy relacionados, pero formalmente distintos, aunque sea frecuente utilizar el mismo nombre para casos concretos y muy importantes. Se trata de los conceptos de “**Tipo de Datos**”, “**Tipo abstracto de Datos**” y “**Estructura de Datos**”. Las definiciones se pueden establecer de la siguiente forma:

Un **Tipo de Datos (TD)** es el conjunto de valores que puede tomar una variable.

Los datos se pueden clasificar en dos tipos:

- **Dato elemental.** Aquellos que pueden ser almacenados en cualquier soporte de información sin necesidad de ninguna estructura especial para su almacenamiento.

Los datos elementales se pueden clasificar en tres grupos:

- a) **Numéricos.** Lo componen los guarismos (0, 1, 2, 3, ..., 9), se utilizan para representación de información de tipo numérico.
- b) **Alfabéticos:** compuestos por los caracteres del alfabeto (a, b, c, ..., x, y, z), se utilizan para representar información no numérica.
- c) **Especiales:** utilizados con fines sintácticos en la mayoría de los casos o con fines aritméticos en otros. Formados por todos los caracteres que no constituyen información por sí solos (¿, ?, =,), (, /, &, %, etc.).

- **Tipos compuestos.** Los más frecuentes son los **arrays**, **listas**, **árboles**, grafos, registros y ficheros.

Los tipos de datos constituyen un primer nivel de abstracción, ya que no se tiene en cuenta cómo se implementan o se representa realmente la información sobre la memoria de la máquina. Para el usuario, el proceso de implementación o representación es invisible.

Un **Tipo Abstracto de Datos (TAD)** representa una abstracción donde se destacan los detalles de la especificación (**el qué**) y se ocultan los detalles de la implementación (**el cómo**).

Así un TAD es representado por su interfaz la cual muestra las operaciones que se pueden realizar sobre ese tipo de datos sin importar la implementación concreta.

Así podemos definir un TDA Lista, un TDA Arbol, un TDA Pila o un TDA Cola, independientemente de las estructuras de datos usadas para su implementación.

Una **Estructura de Datos (ED)** es una forma de organizar un **conjunto de datos elementales con el objetivo** de facilitar la manipulación de estos datos como un todo o individualmente, es decir, es la representación o implementación de un TAD.

Clasificación de estructuras de datos

1ª CLASIFICACIÓN: Según cómo se ubiquen en la memoria del ordenador:

- **Contiguas:** son aquellas que al representarse en el hardware del ordenador, lo hacen situando sus datos en áreas adyacentes de memoria. Ejemplos: cadenas, arrays, vectores, registros.
- **Enlazadas:** sus datos no tienen por qué situarse de forma contigua en la memoria, en las estructuras enlazadas los datos se relacionan unos con otros mediante punteros. La localización del dato no es inmediata, sino que se produce a través de los punteros que relacionan unos datos con otros.

2ª CLASIFICACIÓN: Según la variabilidad de su tamaño durante la ejecución de un programa:

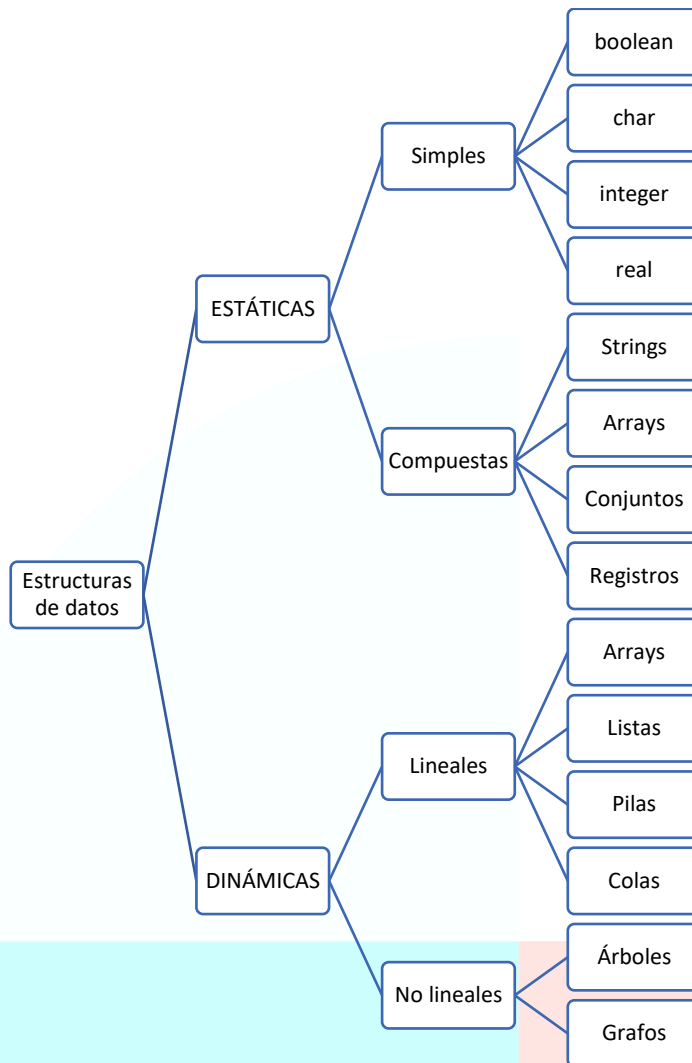
- **Estáticas:** son aquellas en las que el tamaño ocupado en memoria se define con anterioridad a la ejecución del programa que las usa (en tiempo de compilación), y su dimensión no puede modificarse durante el proceso. Ejemplo: array o matriz.
- **Dinámicas:** las estructuras de los datos pueden crecer o no en tamaño, durante la ejecución, dependiendo de las necesidades de la aplicación. Ejemplo: lista o árbol.

Ventajas e inconvenientes de las agrupaciones estáticas frente a las agrupaciones dinámicas:

- En agrupaciones estáticas es más fácil y rápido realizar búsquedas en vectores.
- En general, para un mismo número de elementos las agrupaciones dinámicas ocuparán más memoria que las estáticas (por cada elemento se tiene también un puntero en las agrupaciones dinámicas).
- En agrupaciones dinámicas es mayor la facilidad para insertar y eliminar elementos.
- Es preferible las agrupaciones dinámicas si el número de elementos va a ser cualquiera.

3ª CLASIFICACIÓN: Según los tipos de datos de contengan:

- **Homogéneas:** son aquellas en las que únicamente se utiliza un tipo de datos.
- **Heterogéneas:** las estructuras que permiten que sus elementos puedan ser de tipos de datos distintos.



1. Arrays

Colección de elementos homogéneos (de igual longitud y tipo) entre los que existe una relación lineal determinada por la posición del elemento dentro de la estructura, también se las conoce con el nombre de Array.

Los arrays son estructuras de datos complejas que se caracterizan porque sus unidades homogéneas de información se ubican en posiciones contiguas de memoria.

Los arrays dependiendo del lenguaje de programación pueden ser estáticos (de tamaño fijo determinado en tiempo de compilación) y dinámicos (el tamaño puede variar en tiempo de ejecución).

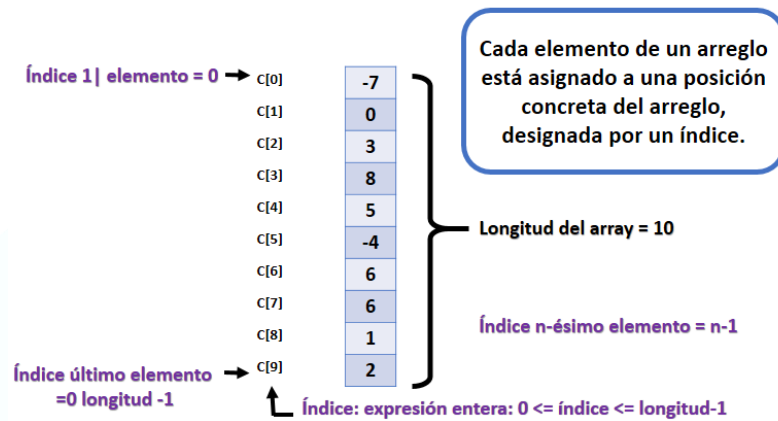
Un array viene definido por:

- La definición de sus elementos. Longitud y tipo de dato a almacenar en la estructura.
- Índice (o varios). Es una variable numérica entera positiva que permite el acceso a los distintos elementos de la estructura según el valor de éste y, por tanto, según la posición

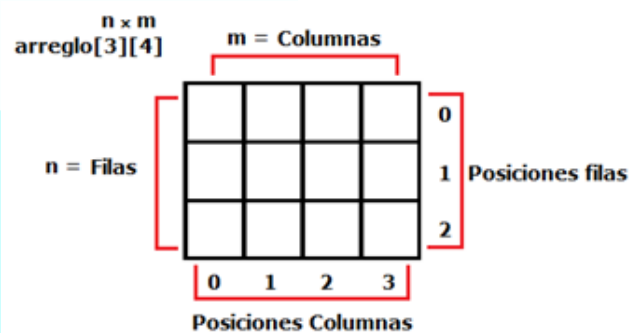
en la que se encuentra el elemento dentro de la estructura. Un índice comienza en **0** y termina en **tamaño-1**.

Podemos distinguir varios **tipos de arrays**:

- **VECTORES** o arrays unidimensionales.

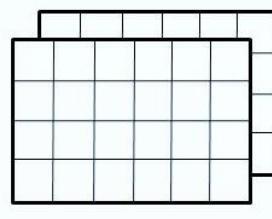


- **MATRICES** o arrays bidimensionales: los elementos quedan estructurados en filas y columnas para su representación gráfica. Para acceder a cada elemento lo haremos con dos índices **i** para las filas y **j** para las columnas.



- **ARRAYS MULTIDIMENSIONALES.**

```
int a[10];      /* Array de una dimension */
int b[3][5];    /* Array de dos dimensiones */
int c[6][4][2]; /* Array de tres dimensiones */
```



Respecto a la implementación del array se establece el número de elementos que forman la estructura, que siempre habrá que fijarlo, de alguna de las siguientes maneras:

- **Memoria estática** (en tiempo de compilación): una vez que el programa ha sido preparado para ser ejecutado (compilado y enlazado), la posición y el espacio de memoria que utilizará el array en la ejecución del programa ya está fijado y no puede de ninguna forma cambiarse, hablamos de array estático.
- **Memoria dinámica** (en tiempo de ejecución): diremos que el array es dinámico, pues el espacio en memoria no es de tamaño fijo, pudiendo el espacio de memoria variar en tiempo de ejecución.

Las operaciones básicas que se realizan sobre un array son: crear el array, insertar/modificar/eliminar elementos, buscar un elemento concreto, ordenar los elementos del array y contar los elementos que contiene.

La estrategia utilizada para la búsqueda y ordenación de elementos de un vector se verá en un apartado posterior.

2. Registros

Las estructuras o registros son agrupaciones heterogéneas, contiguas y estáticas.

Los elementos contenidos en la agrupación pueden ser de distintos tipos y se denominan campos, y suelen estar relacionados con la información referente a un objeto concreto.

Ejemplos:

Una persona (Nombre, edad, DNI...)

Un libro (Título, autor, precio, disponible/agotado...)

Los registros o estructuras definen nuevos tipos de datos que podemos utilizar a lo largo de nuestro programa.

Es decir, una vez declarada la estructura podemos utilizar y declarar variables de ese nuevo tipo (de la misma forma que si declarásemos variables de tipo entero o real).

La manera de acceder a cada uno de los campos es mediante el operador de acceso '.'.

Ejemplo:

```
//Definición registro
struct Nombre {
    Tipo1 Campo1; Tipo2 Campo2; ...
};
```

```
//Declaración variable
NombreVar : Nombre
//Asignación
NombreVar.Campo1 = valor;
```

Los registros tienen como característica el **anidamiento de estructuras**. Las estructuras se pueden anidar, es decir, uno de los campos de una estructura puede ser a su vez otra estructura.

3. Listas

Una lista es un TAD que se define como un conjunto lógico de elementos **homogéneos** (nodos) entre los que existe una **relación lineal**, considerados a cada uno de éstos como unidad básica de información dentro de la estructura. Cada elemento de la estructura puede estar formado por uno o varios campos.

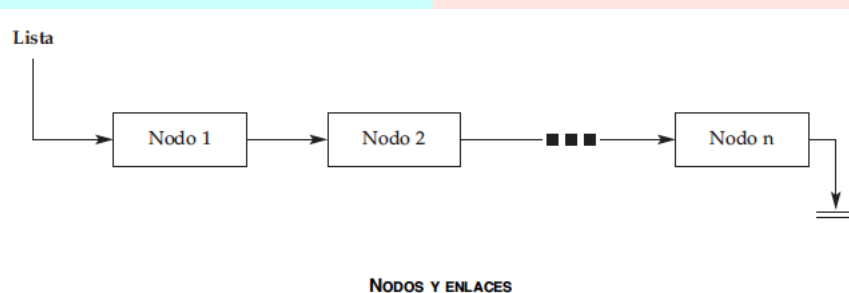
Nos permite almacenar datos de una forma organizada, al igual que los vectores pero, a diferencia de estos, esta estructura es dinámica, por lo que no tenemos que saber "a priori" los elementos que puede contener. Es posible utilizar una estructura estática aunque no es lo habitual.

Lista = colección de elementos homogéneos entre los que existe una relación lineal:

- Cada elemento de la lista, a excepción del **primero**, tiene un único predecesor.
- Cada elemento de la lista, a **excepción del último**, tiene un único sucesor.

Una lista es una estructura de datos que cumple:

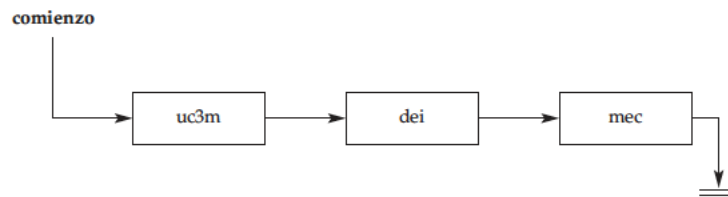
- Todos sus componentes son del mismo tipo.
- Cada elemento o nodo (datos+puntero) va seguido de otro del mismo tipo o de ninguno.
- Sus componentes se almacenan según cierto orden.



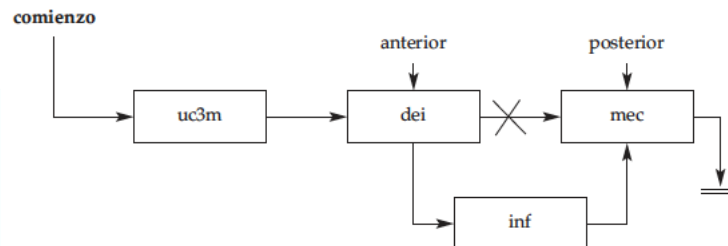
Las **operaciones** sobre una lista son: crear, insertar un elemento, buscar un elemento (si la lista es dinámica solo es posible la búsqueda secuencial) y eliminar un elemento.

De entre estas operaciones la de mayor complejidad técnica es la de inserción, existiendo 3 casuísticas.

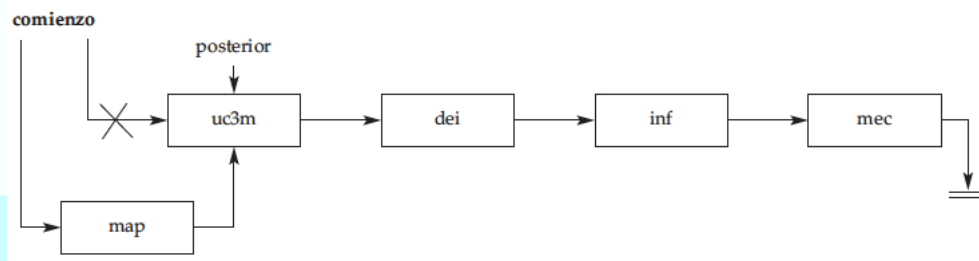
Ejemplo: Partimos de la siguiente lista:



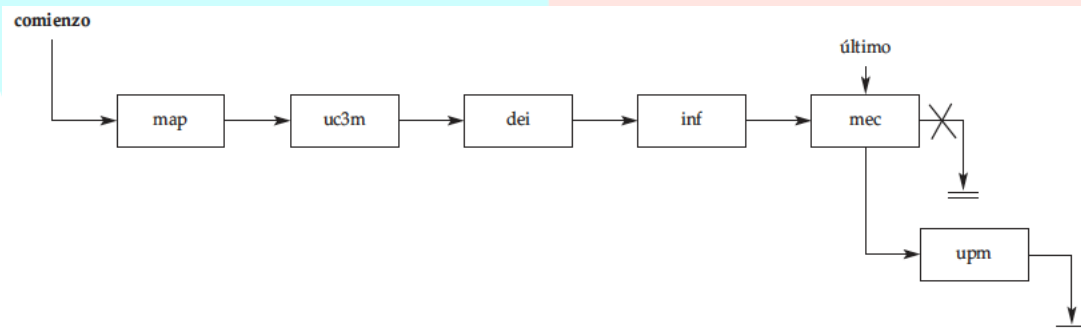
Inserción de un elemento en mitad de la lista:



Inserción de un elemento al principio de la lista:



Inserción de un elemento al final de la lista:



Tipos de listas según la forma de acceder al siguiente elemento:

- **Lista Densa o CONTIGUA:** la propia estructura determina cuál es el siguiente elemento de la lista (posición), los **nodos son adyacentes en memoria**. Por ejemplo, se puede usar una tabla para implementar una lista, de forma que el siguiente elemento de la lista fuera dado por el orden del índice (no confundir con el orden de los elementos del array). Nótese sin embargo que, puesto que un array es estático, la memoria reservada para implementar esta "lista" también será estática.

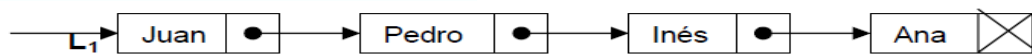
- **Lista ENLAZADA:** cada elemento contiene la información necesaria para llegar al siguiente. Es una estructura dinámica, por lo que no tenemos que saber "a priori" los elementos que puede contener.

En una lista enlazada, cada elemento apunta al siguiente excepto el último que no tiene sucesor y el valor del enlace es null.

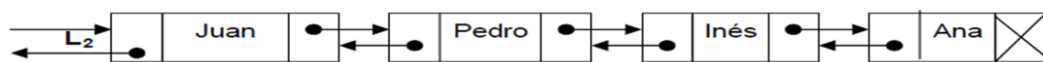
La posición del siguiente elemento de la estructura la determina el elemento actual. Es necesario almacenar al menos la posición de memoria del primer elemento. Además, como es dinámica, su tamaño cambia durante la ejecución del programa.

Dentro de las listas enlazadas podemos distinguir, según la cantidad de enlaces por nodo, entre:

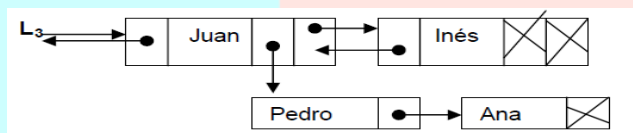
- **Lista simplemente enlazada:** cada elemento conoce que elemento es el que le sucede en el orden, pero no cual le precede.



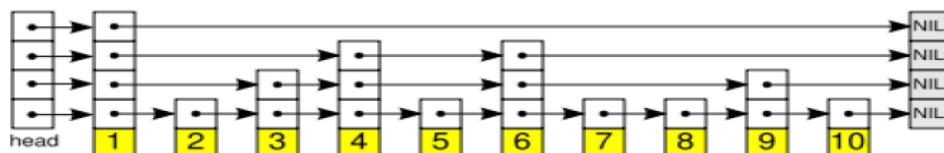
- **Lista doblemente enlazada:** cada elemento conoce que elemento le precede y cual le sucede en el orden. Permite el recorrido de la lista en ambos sentidos. Sin embargo, como inconveniente ocupa más memoria que una lista simplemente enlazada.



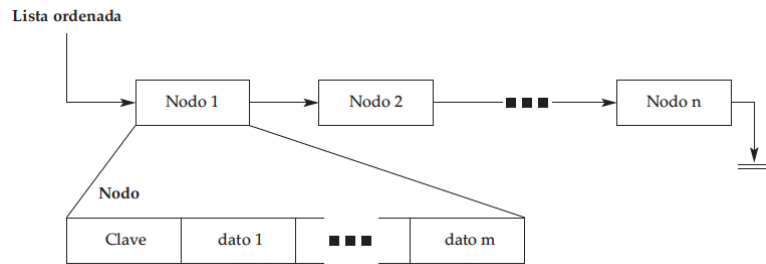
- **Lista con enlaces múltiples:** son aquellas listas en donde cada elemento, aparte de conocer los elementos que le preceden o suceden, también tiene uno o varios enlaces al primer elemento de otra lista, siendo éstas sublistas de la anterior.



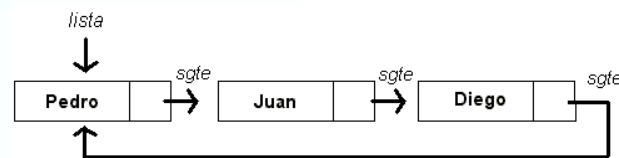
- **Skip list o lista por saltos:** es una estructura de datos, basada en listas enlazadas paralelas con **eficiencia** comparable a la de un árbol binario. Es una lista ordenada (y en principio sin claves repetidas) en la que cada nodo puede tener uno o varios enlaces al nodo siguiente y a diversos **nodos a distancias variables**, bien establecidas de forma aleatoria o siguiendo un patrón establecido de saltos a longitudes fijas.



- **Lista ORDENADA**: la posición de cada nodo viene determinada por el **valor** de uno o más campos obligatorios de información del nodo denominados **clave**. No se permite tener dos nodos con la misma clave.

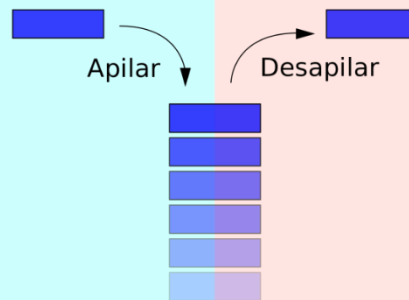


- **Lista CIRCULAR**: es una lista en la que el **último nodo apunta al primero** en lugar de ser null. Una lista circular se puede aplicar a listas simplemente enlazadas como a listas doblemente enlazadas.



4. Pilas

Una pila se considera un **caso particular lista**. Es un TAD que se define como una colección de elementos homogéneos en la que **sólo se pueden añadir y eliminar elementos por el principio** de la misma (cabecera o tope), siguiendo la filosofía **LIFO¹ (Last In First Out)**.



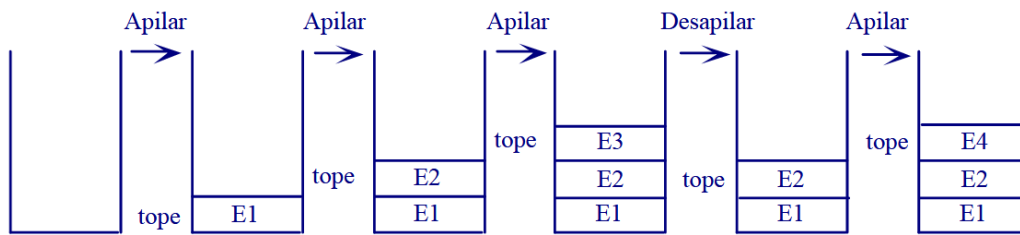
Un concepto a destacar es la **cabecera o el tope** que **apunta a la posición** en la que se **insertan y eliminan los elementos de la pila**.

¹ En la teoría de colas, encargada del estudio de las colas o las líneas de espera, los **modelos de disciplina de cola** se refieren a la forma de establecer el orden en el que sus miembros (elementos) se seleccionan para recibir el servicio. Destacan los siguientes modelos:

- **LIFO (Last In First Out)**: se atiende **primero** al miembro que ha llegado el **último**.
- **FIFO (First In First Out)**: se atiende **primero** al miembro que ha llegado antes.
- **RSS (Random Selection of Service)**: se atiende a los miembros de forma **aleatoria**.

Las operaciones básicas para manipular una pila son:

- **Push o Apilar**: añade un elemento en la posición siguiente al tope actual.
- **Pop o Desapilar**: elimina el elemento situado en el tope.



Aplicaciones prácticas de una pila:

- **Vuelta atrás en un navegador web.**
- Función **Deshacer** en un procesador de texto.
- **Invocación a métodos.**

```
main(){
  int i = 5;
  fno(i);
}
fno(intj) {
  int k;
  k = j+1;
  bar(k);
}
bar(int m){
  ...
}
```

bar
PC = 1
m = 6
fno
PC = 3
j = 5
k = 6
main
PC = 2
i = 5

- Cuando se invoca un método, se inserta en la pila una nueva entrada.
- Por cada método se guardan:
 - Variables locales.
 - Valor devuelto.
 - Contador de programa.
- Cuando el método finaliza se saca ese elemento de la pila y se devuelve el control al método que esté en la cabecera.

- **Notación polaca inversa** (RPN, Reverse Polish Notation): es otra forma de escribir expresiones matemáticas (lo normal, notación infija). Se usa en ciertas calculadoras. Por ejemplo, dada la expresión infija $10-(4+3)*2$ (el resultado es -4), su equivalente en **notación postfija** (o postfix) es $10\ 4\ 3\ +\ 2\ *\ -$. Forma de resolver: recorrer la expresión de izquierda a derecha, apilando los números (operandos). Cuando se encuentra un operador se desapilan los dos últimos operandos, se resuelve la operación y el resultado se apila de nuevo. Finalizada la expresión, en la pila solo debe quedar el resultado final. Vemos el ejemplo:

10 4 3 + 2 * -

Se inserta 10

10

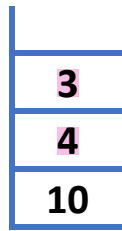
10 4 3 + 2 * -

Se inserta el 4

4
10

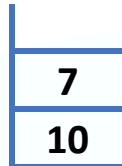
10 4 **3** + 2 * -

Se inserta el 3



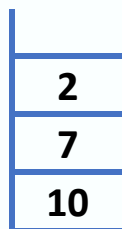
10 4 3 **+** 2 * -

Se suman 3 y 4 y se apila el resultado



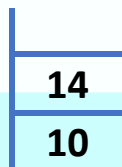
10 4 3 + **2** * -

Se inserta el 2



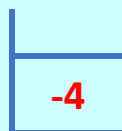
10 4 3 + 2 ***** -

Se multiplican 2 y 7 y se apila el resultado



10 4 3 + 2 * **-**

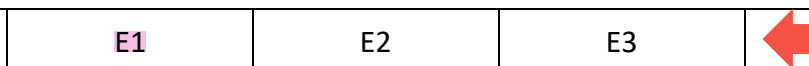
Se restan 14 y 10 y se apila el resultado



5. Colas

Una cola también es considerada un caso especial de lista. Es un TAD que se define como una colección ordenada de elementos homogéneos en la que sólo se pueden añadir elementos por el final y se eliminan por el principio (frente), siguiendo la filosofía **FIFO** (First In First Out).

Desencolar



Encolar

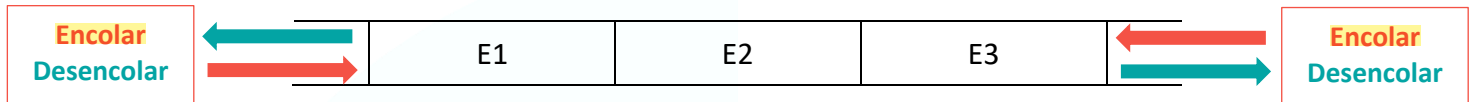
Las operaciones básicas para manipular una cola son:

- **Queue o Encolar**: añade un elemento al final de la cola.
- **Dequeue o Desencolar**: elimina el primer elemento de la cola.

Aplicaciones prácticas de una cola:

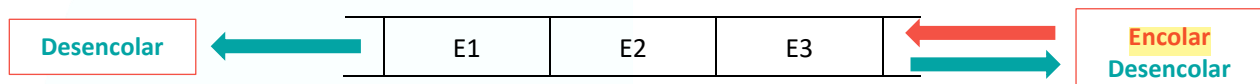
- Listas de **espera**.
- Acceso a recursos compartidos dedicados.
- Multiprogramación de la CPU.

Variante: **bicola** (cola **doble**) es una cola en la que los elementos pueden ser insertados o borrados por cualquiera de los dos extremos.

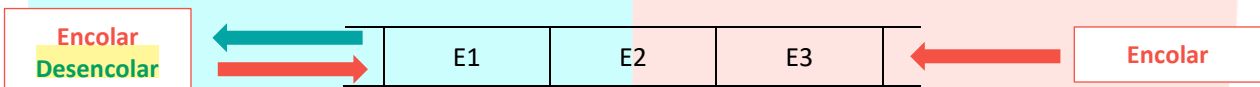


A su vez, las bicolas ofrecen dos variantes:

- **Bicola de entrada restringida**: las inserciones solo se hacen por un extremo (el extremo final) y las eliminaciones por cualquiera de los dos extremos.



- **Bicola de salida restringida**: las inserciones se hacen por cualquiera de los dos extremos y las eliminaciones solo por un extremo (el extremo inicial).



Variante: **cola de prioridades** es una cola en la que cada **elemento** tiene asignada una **prioridad**. Un elemento con mayor prioridad será desencolado antes que un elemento de menor prioridad. Si dos elementos tienen la misma prioridad, se desencolará siguiendo el **orden de entrada**.

6. Conjuntos

Es una colección homogénea y no ordenada de elementos distintos, entre los cuales existe una relación de orden.

Se suele implementar mediante arrays.

Variante: TAD **Bolsa** (multiconjunto) es un conjunto en el que se permiten elementos **repetidos**.

7. Mapas

Un **mapa** (array asociativo) es una **generalización** del concepto de **conjunto**, formado por un **conjunto** de **pares** {clave, valor}, en la cual cada elemento que pertenece al conjunto (denominado clave) tiene asociado un valor.

Como restricción principal, las **claves no pueden estar repetidas** (los valores sí). Los elementos se localizan mediante su clave, teniendo en cuenta que **no existe una relación de orden predefinido**.

8. Diccionarios

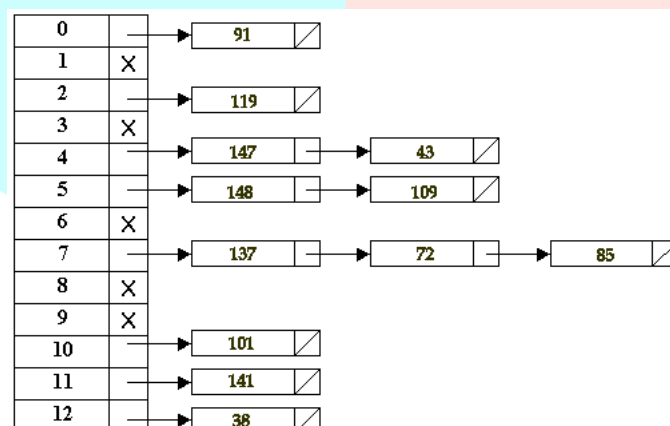
Un diccionario es un **mapa** para que el que se define una **relación de orden sobre las claves**.

9. Tabla hash

Una **tabla Hash** o tabla de **dispersión** es un contenedor asociativo (tipo **Diccionario**) que permite el almacenamiento y posterior recuperación eficientes de elementos (denominados valores) a partir de otros objetos, llamados claves.

Una tabla hash **se puede ver como un conjunto de entradas** o pares del tipo {clave, valor}. Cada una de estas entradas tiene asociada una clave única y, por lo tanto, diferentes entradas de una misma tabla tendrán diferentes claves (no pueden existir claves repetidas). Esto implica, que una clave identifica unívocamente a una entrada en una tabla hash.

Una **tabla hash está formada por un array de entradas**, que será la estructura que almacene la información, y por una función de dispersión. La **función de dispersión** permite asociar el **elemento almacenado** en una entrada con la **clave de dicha entrada**. Por lo tanto, es un algoritmo crítico para el buen funcionamiento de la estructura.



Cuando se trabaja con tablas hash es frecuente que se produzcan **colisiones**. Las **colisiones** se producen cuando para **dos elementos de información distintos**, la función de dispersión

les asigna la misma **clave**. Como se puede suponer, esta solución se debe arreglar de alguna forma. Para ello las tablas hash cuentan con una **función de resolución de colisiones**.

Existen dos tipos de tablas hash, en función de cómo resuelven las colisiones:

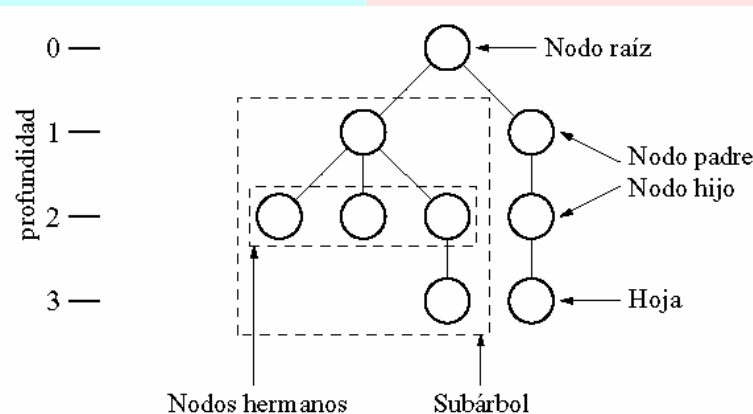
- **Encadenamiento separado (hashing ABIERTO)**: las colisiones se resuelven insertándolas en una lista. De esa forma tendríamos como estructura un vector de listas. Al número medio de claves por lista se le llama **factor de carga** y habría que intentar que esté **próximo a 1**.
- **Direccionamiento abierto (hashing CERRADO)**: utilizamos un vector como representación y cuando se produzca una colisión la resolvemos **reassignándole otro valor hash a la clave hasta que encontremos un hueco**.

10. Árboles

Un árbol es una estructura de datos **dinámica, jerárquica y no lineal**, en la que existen relaciones padre-hijo entre nodos. Formalmente es un **grafo no dirigido acíclico y conexo**.

Se caracterizan por estar formados por una serie de nodos conectados por una serie de aristas verificándose que:

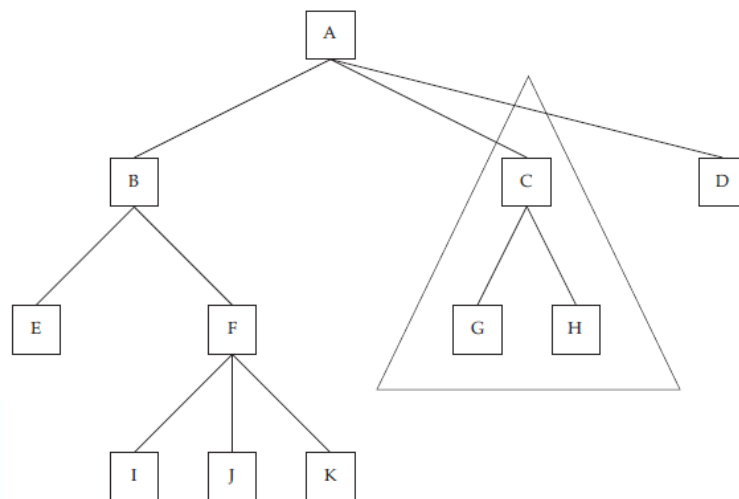
- Hay un **único** nodo raíz.
- Cada nodo, excepto el nodo raíz, tiene **un único padre**.
- Hay un **único camino** desde la raíz hasta cada nodo, es decir, **no hay ciclos**.



Y como definiciones básicas tenemos que:

- El **nodo raíz** es el **único nodo sin padre**.
- Un **nodo interno** es aquel que tiene al menos un hijo.
- Un **nodo hoja** es aquel **nodo que no tiene descendientes**.

A partir del árbol del ejemplo se expondrán algunas definiciones:



- El **grado de un nodo** es el número de **hijos o descendientes directos**.
En el ejemplo, $\text{Grado}(F)=3$, $\text{Grado}(B)=2$ y $\text{Grado}(A)=3$.
- El **grado de un árbol** es el **mayor grado** de sus nodos.
En el ejemplo, $\text{Grado}=3$.
- Un **subárbol** es el árbol formado por un nodo y todos sus descendientes.
En el ejemplo, el formado por $\{C, G, H\}$.
- La **profundidad de un nodo** es la longitud del camino desde la raíz hasta ese nodo.
En el ejemplo, $\text{Profundidad}(J)=3$, $\text{Profundidad}(D)=1$ y $\text{Profundidad}(H)=2$.
- La **altura de un árbol** es la profundidad máxima de cualquier nodo.
En el ejemplo, $\text{Altura}=3$.
- El **nivel de un nodo** se define según este criterio: la raíz del árbol tiene nivel 1, el nivel de cualquier nodo es el nivel de su padre más uno. El nivel de un árbol vacío es cero.
En el ejemplo, el árbol tiene 4 niveles, el nivel de A es 1 y el nivel de G es 3.
- Un **camino** entre dos nodos X e Y existe si existe una sucesión de nodos que permitan llegar desde X a Y.
En el ejemplo, $\text{Camino}(A, K) = \{A, B, F, K\}$ y $\text{Camino}(C, K) = \{\}$.
- Un **árbol ponderado** es aquel en el que cada **arista** tiene asociado un **peso**.

Por último, el TAD Lista es un árbol degenerado de grado 1.

Por otro lado, las **operaciones** básicas a realizar con los árboles son: crear un árbol, recorrer el árbol (en amplitud o en profundidad), insertar/eliminar un nodo, buscar un elemento, **ordenar** el árbol y contar número de nodos.

Respecto a la implementación, los árboles pueden implementarse internamente mediante arrays o punteros. Lo más habitual es utilizar memoria dinámica, es decir, **punteros**.

Clasificación de los árboles según el grado

Según el grado del árbol podemos clasificar los árboles en:

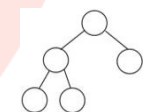
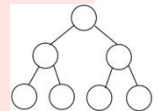
- **Árboles BINARIOS**: de grado = 2.
 - Árbol binario de búsqueda (ABB).
 - Árbol AVL: ABB equilibrado en altura.
- **Árboles MULTICAMINO**: de grado > 2.
 - Árbol B.
 - Árbol B⁺.
 - Árbol B*.

Árboles BINARIOS (AB)

Un árbol binario es un árbol de grado 2, tiene un nodo raíz y se cumple que cada nodo tiene como máximo dos hijos: el **hijo izquierdo** y el **hijo derecho**.

Definiciones de árboles binarios:

- **Árbol binario lleno**: AB donde todo nodo interno (excepto las hojas) tiene 2 hijos.
- **Árbol binario completo**: AB lleno en todos los niveles excepto el último, que deberá estar **ocupado (sin huecos)** de izquierda a derecha.

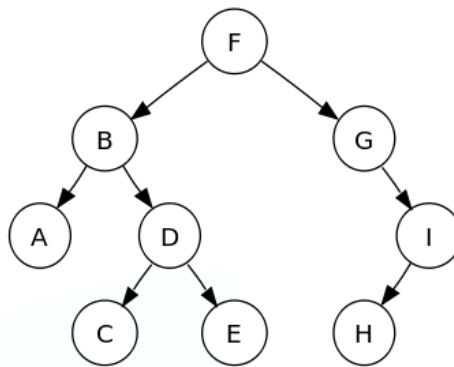


¿Cómo recorrer los nodos de un árbol? Dos tipos de recorrido: en **profundidad** y en **anchura**.

Recorrido en PROFUNDIDAD: dimensión **vertical**. Existen 3 modos:

- **Preorden (R-IZQ-DCHA)**: primero se visita el **nodo raíz**, luego el subárbol izquierdo y, por último, el subárbol derecho.
- **Inorden (IZQ-R-DCHA)**: primero se visita el **subárbol izquierdo**, luego el **nodo raíz** y, por último, el subárbol derecho.
- **Postorden (IZQ-DCHA-R)**: primero se visita el subárbol izquierdo, luego el subárbol derecho y, por último, el **nodo raíz**.

Ejemplo: Recorrido en profundidad del árbol siguiente.



Preorden: F, B, A, D, C, E, G, I, H.

Inorden: A, B, C, D, E, F, G, H, I.

Postorden: A, C, E, D, B, H, I, G, F

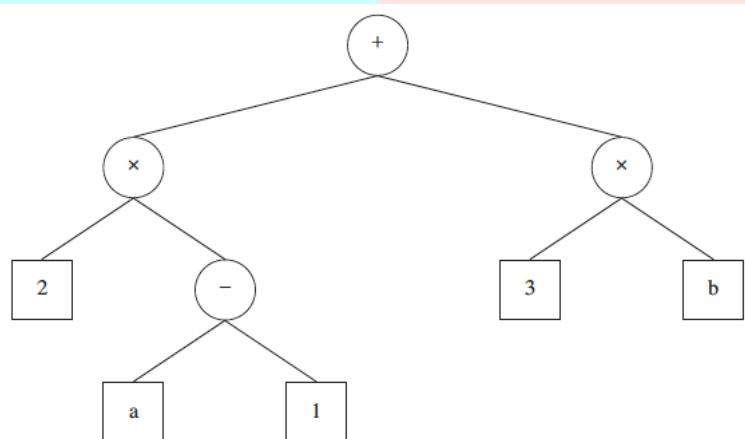
Recorrido en ANCHURA: dimensión horizontal. Se recorre el árbol por niveles y de izquierda a derecha.

Ejemplo: Recorrido en anchura del árbol del ejemplo anterior.

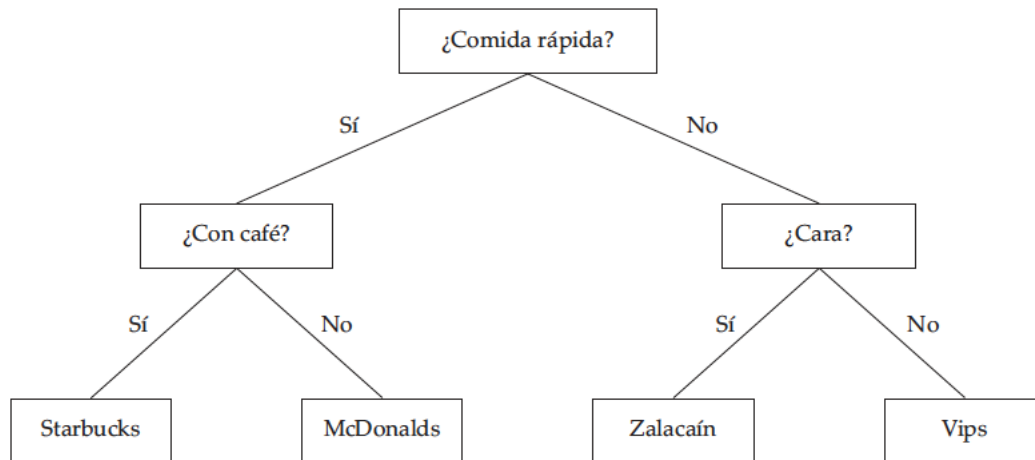
F, B, G, A, D, I, C, E, H

Las aplicaciones prácticas de este tipo de estructura son diversas, pudiendo destacar las expresiones aritméticas y los árboles de decisión.

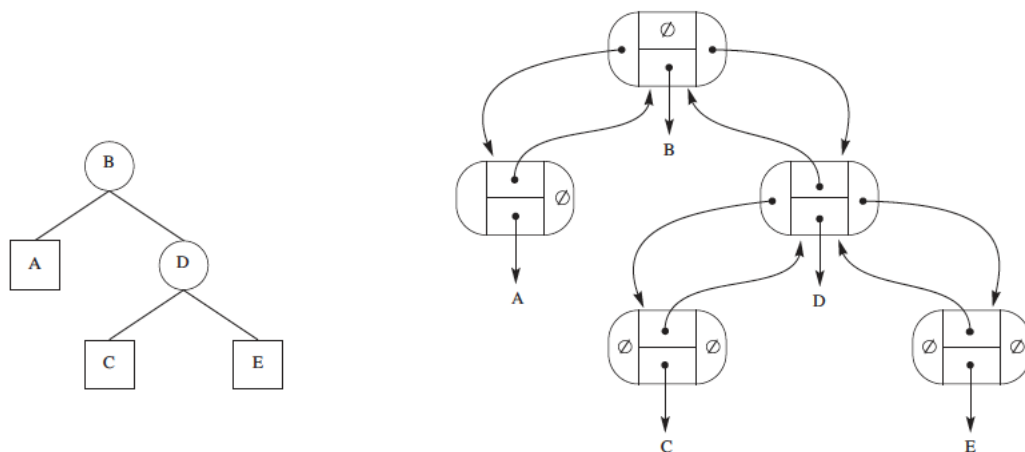
Ejemplo: expresiones aritméticas. $2(a-1)+3b$



Ejemplo: árboles de decisión. El nodo interno contiene preguntas con respuesta Sí/No y los nodos hoja decisiones.



La estructura de datos de un árbol binario es dinámica, donde cada nodo contiene un elemento, un puntero al nodo hijo izquierdo, un puntero al nodo hijo derecho y, quizá, un puntero al nodo padre.



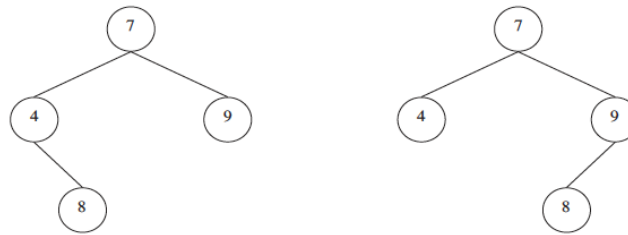
Árboles BINARIOS DE BÚSQUEDA (ABB)

En este tipo de árboles cada nodo tiene asociado un valor clave y este valor de clave es mayor que los valores de clave de los nodos de su subárbol izquierdo y menor que todos los de su subárbol derecho.

Claves Subárbol IZQUIERDO < Clave nodo RAÍZ < Claves Subárbol DERECHO

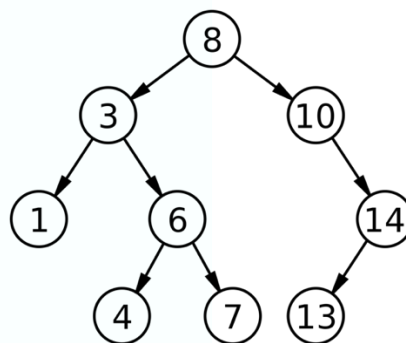
Esto significa que para que un árbol binario sea ABB, se tiene que cumplir para cualquier nodo, que todos los valores del subárbol izquierdo del nodo son menores que el valor del nodo y que todos los valores del subárbol derecho. deben ser mayores que los valores del nodo.

Ejemplo: ¿Cuál de los siguientes árboles es un ABB?



Una de las ventajas de utilizar un ABB es que si se realiza un recorrido en Inorden por un ABB accederemos a los valores en orden ascendente. El proceso de búsqueda en un árbol binario de búsqueda se realiza recursivamente, desde la raíz hasta las hojas.

Ejemplo: Recorrido Inorden de un ABB.



Inorden (I-R-D) = 1,3,4,6,7,8,10,13,14

Un inconveniente de esta estructura es que puede llegar a degenerar en una lista, se soluciona equilibrando el ABB.

Árboles AVL

Un árbol AVL (Adelson-Velskii y Landis) es un ABB **equilibrado en altura**.

La condición que debe satisfacer un árbol para que sea AVL es que **para todo nodo la altura de sus subárboles izquierdo y derecho no difiere en más de una unidad**.

ABB AVL:

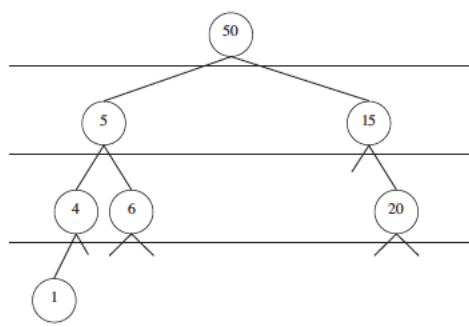
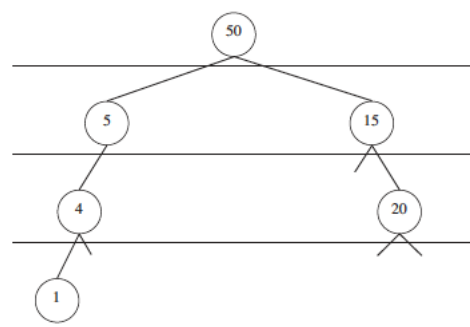


ABB no AVL:



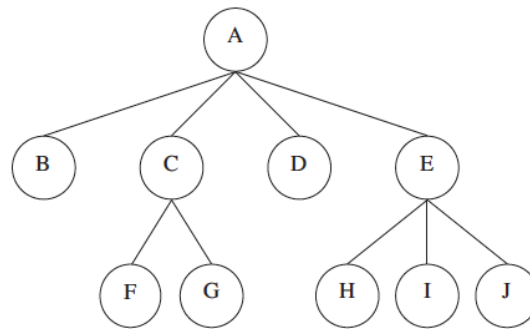
Gracias a que el árbol es equilibrado las búsquedas son más rápidas: $O(\log N)$.

Cada nodo, además de la información que se pretende almacenar, debe tener los dos punteros a los árboles derecho e izquierdo, igual que los árboles binarios de búsqueda (ABB), y además el dato que controla el factor de equilibrio.

Se puede imponer una restricción mayor si se pretende que el árbol tenga un **equilibrio perfecto**. La condición consiste en que, para todo nodo del árbol, el número de nodos del subárbol izquierdo no difiere en más de una unidad del número de nodos del subárbol derecho.

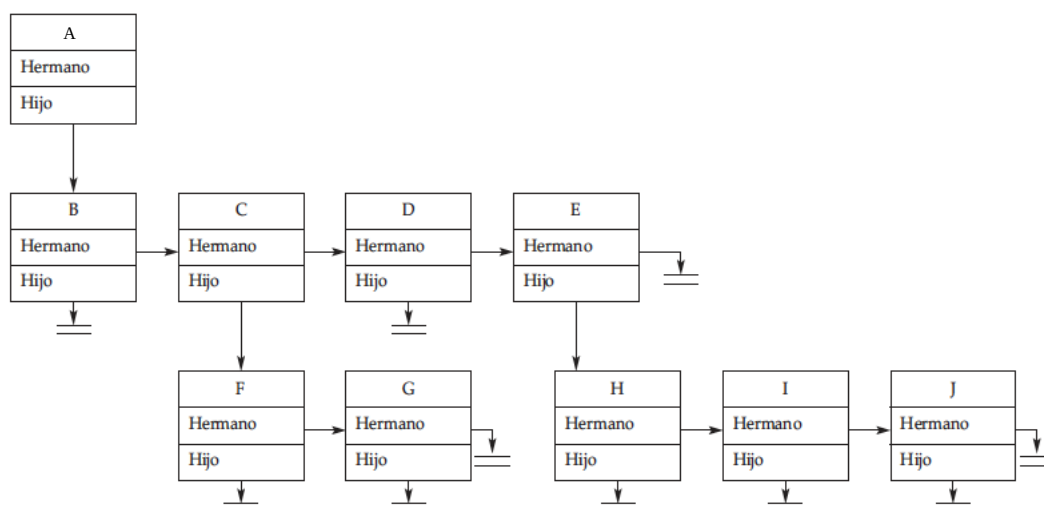
Árboles multicamino

Son aquellos cuyo grado es mayor que 2. Se refiere el multicamino a que cada nodo tiene un número arbitrario de nodos. Una implementación típica es tener una lista de hijos por cada nodo.



La implementación de los árboles multicamino se puede abordar de dos formas:

- Diseñando los hijos como arrays de punteros: desaprovecha la memoria si el número de hijos es muy variable a no ser que se diseñe de manera dinámica y no debería de usarse si el número de hijos es ilimitado.
- Diseñando los hijos con punteros al hermano siguiente y a su hijo.



La principal ventaja de este tipo de árboles consiste en que existen más nodos en un mismo nivel que en los árboles binarios con lo que se consigue que, si el árbol es de búsqueda, los accesos a los nodos sean más rápidos.

Por el contrario, el inconveniente más importante que tienen es la mayor ocupación de memoria, pudiendo ocurrir que en ocasiones la mayoría de los nodos no tengan descendientes o al menos no todos los que podrían tener desaprovechándose por tanto gran cantidad de memoria.

Árboles B

Son árboles multicamino, de un orden determinado que se caracteriza por:

- Cada **página** tiene como máximo **N nodos**.
- Cada **página excepto la raíz** tiene **como mínimo N/2 nodos**.
- Cada **página es o una página hoja o tiene m+1** descendientes (m es el número de nodos que contiene una página).

Se define el **orden de un árbol B** como el número máximo de hijos de una página.

La estructura de una página con m claves es:

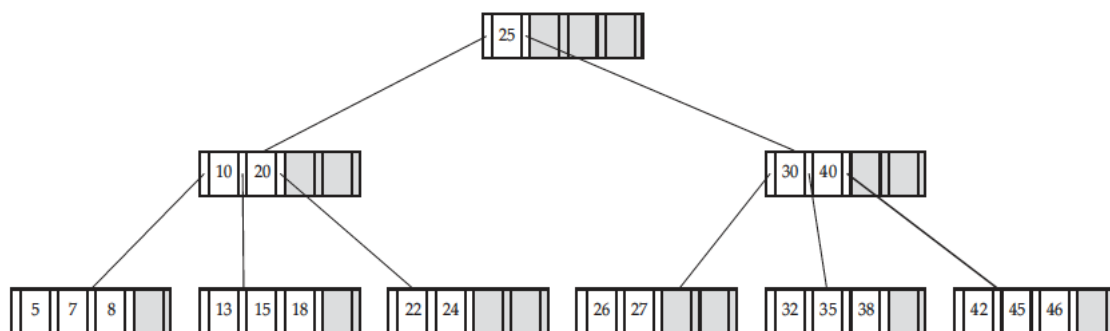
p_0	k_1	p_1	k_2	p_2	...	k_m	p_m
-------	-------	-------	-------	-------	-----	-------	-------

donde se cumple siempre que:

$$k_i < k_{i+1} \quad \forall i = 1 \dots m$$

$$N/2 \leq m \leq N$$

$\forall i = 1 \dots m, p_i$ apunta a una página donde $k_i \leq \text{claves página} < k_{i+1}$.



Notar que los árboles B **crecen “hacia arriba”** y decrecen **“hacia abajo”** y que las hojas siempre están al mismo nivel.

Como ventaja de esta estructura de datos es que se garantiza una ocupación en cada página de al **menos el 50%**.

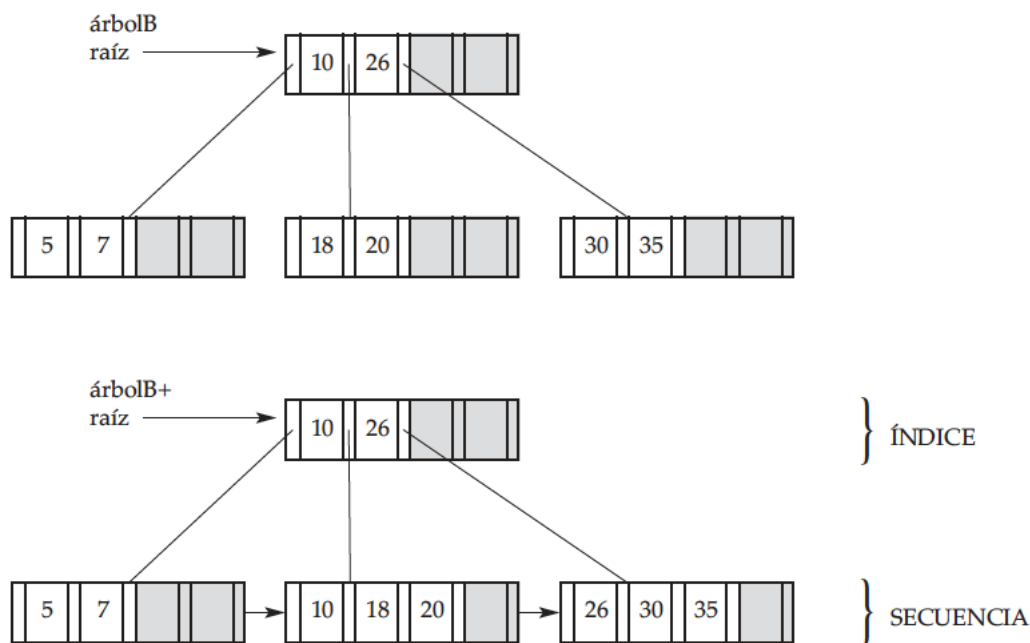
Como aplicación práctica, podemos encontrar que los árboles B son estructuras utilizadas en bases de datos y sistemas de archivos.

Árboles B⁺

Es un árbol con las mismas características que los árboles B, que está formado por dos partes:

- **Índice:** nodos interiores.
- **Secuencia:** páginas hoja enlazadas secuencialmente en las que se repiten las claves interiores.

Todas las hojas se encuentran en el mismo nivel, que corresponde al más bajo. Los nodos hoja se encuentran unidos entre sí como una lista enlazada para permitir búsqueda secuencial.



Árboles B^{*}

Este tipo de estructura mejora la eficiencia del acceso directo y la sobrecarga de reorganizar el árbol en la inserción y el borrado.

Como característica principal se asegura la ocupación de cualquier nodo excepto el nodo raíz sea de **al menos 2/3**, en lugar del 50% de los árboles B.

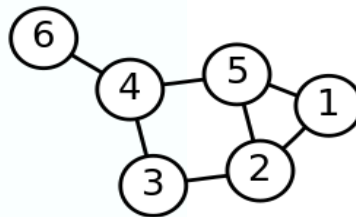
No se debe confundir un árbol B^{*} con un árbol B⁺, en el que los nodos hoja del árbol están conectados entre sí a través de una lista enlazada, aumentando el coste de inserción para mejorar la eficiencia en la búsqueda.

11. Grafos

Es un conjunto de objetos llamados **vértices o nodos** unidos por enlaces llamados **aristas o arcos**.

Típicamente, un grafo se representa gráficamente como un conjunto de puntos (vértices nodos) unidos por líneas (aristas).

Formalmente, un grafo consiste en un conjunto de vértices o nodos y un conjunto de arcos. Se representa con el par $G = (V, E)$. Un arco o **arista** está formado por un par de nodos y se escribe (u,v) siendo u, v el par de nodos.



DEFINICIONES sobre grafos

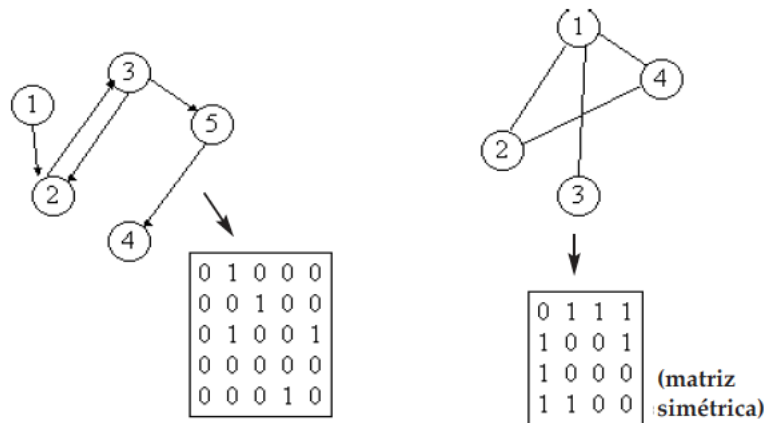
- El **orden** de un grafo es el número de **vértices**.
- El **tamaño** de un grafo es el número de **aristas**.
- Un grafo es **dirigido** si los pares de nodos que forman los arcos son ordenados, se representan $u \rightarrow v$. A este tipo de grafo se le denomina también **digrafo**.
- Un grafo **no dirigido** es aquel que los arcos están formados por pares de nodos no ordenados, se representa $u - v$.
- Un grafo no dirigido es **conexo** si existe un camino entre cualquier par de nodos que forman el grafo.
- Un grafo es **doblemente conexo** si cada par de vértices está conectado por al menos dos caminos disjuntos.
- Un grafo es **trivial** si tiene **un vértice** y ninguna arista.
- Un grafo es **nulo** si no tiene vértices ni aristas.
- Un grafo es **vacío** si no tiene aristas.
- Un grafo **simple** es aquel que no tiene **bucles** ni aristas **múltiples**, es decir, solo se acepta una sola arista uniendo dos vértices cualesquiera.
- Si existe un arco desde un vértice a sí mismo (u, u) , el **camino $u - u$** se denomina **bucle**.
- Un grafo con **bucles** se denomina **pseudografo**.

- Un grafo que no es simple se denomina **multigrafo**, es decir, se permite más de una arista entre dos vértices.
- Un grafo es **completo** si cada par de vértices está unido por una arista.
- Un grafo es **bipartito** si los vértices del grafo se pueden separar en dos conjuntos disjuntos (W y X) y cada arista tiene un vértice en W y otro en X , es decir, las aristas no relacionan vértices del mismo conjunto.
- Si (u, v) es una arista, entonces u y v se dice que son **vértices adyacentes**. Si (u, v) es una arista dirigida, entonces u se dice que es adyacente a v y v se dice que es adyacente de u .
- Un arco tiene, a veces, asociado un factor de peso, en cuyo caso se dice que es un grafo valorado o **ponderado**.
- En un grafo no dirigido el **grado** de un nodo u , $\text{Grado}(u)$, es el número de aristas que contienen a u .
- En un grafo dirigido se distingue entre **grado de entrada** y **grado de salida**. El grado de entrada de un nodo u , $\text{Grado}_{\text{ent}}(u)$, es el número de arcos que llegan a u , y el grado de salida de u , $\text{Grado}_{\text{sal}}(u)$, es el número de arcos que salen de u .
- Un grafo es **regular** si todos los nodos tienen el mismo grado.
- Un **camino** P , en el grafo G , de longitud n desde un vértice u_0 a un vértice u_n es la secuencia de $n + 1$ vértices.
- Un **camino** $P=(u_0, u_1, u_2, \dots, u_n)$ es **simple** si todos los nodos que forman el camino son distintos, pudiendo ser iguales los extremos del camino.
- Un camino es **euleriano** si es simple y además contiene a todas las aristas del grafo.
- Un **ciclo** o **circuito** es un camino simple cerrado donde $u_0=u_n$ y está compuesto al menos por tres nodos (empieza y acaba en el mismo nodo).
- Un circuito o **ciclo hamiltoniano** es un ciclo simple que contiene todos los vértices.
- Un grafo es **acíclico** si no tiene ciclos. Se le denomina también **bosque**.
- Un **árbol de expansión** o **árbol recubridor** o **árbol generador** es, dado un grafo conexo y no dirigido, un subgrafo formado por un árbol que contiene todos los vértices del grafo inicial (puede haber varios).
- Un **árbol de expansión mínimo** o **árbol recubridor mínimo** o **árbol generador mínimo** es, dado un grafo conexo, ponderado y no dirigido, un subgrafo formado por un árbol que contiene todos los vértices del grafo inicial tal que el coste es mínimo.

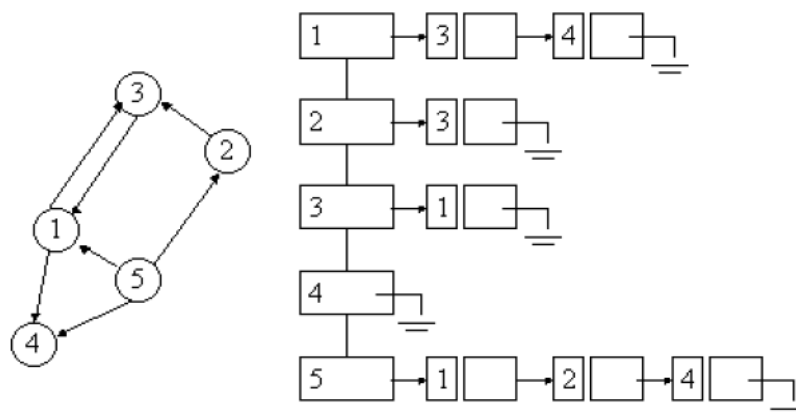
Con las definiciones expuestas podemos redefinir el concepto de **árbol** como un **grafo no dirigido, acíclico y conexo**.

REPRESENTACIÓN de un grafo

- Mediante arrays: **matriz de adyacencia**. Los nodos se representan del 1 al n y los arcos en una matriz $n \times n$, donde el elemento a_{ij} es 1 si existe una arista en (u_i, u_j) y 0 si no existe.



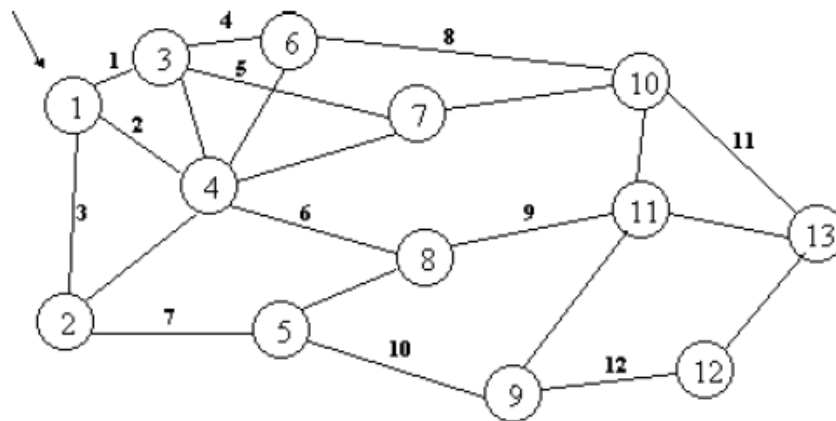
- Mediante estructuras multienlazadas: **lista de adyacencia**.



RECORRIDO de un grafo

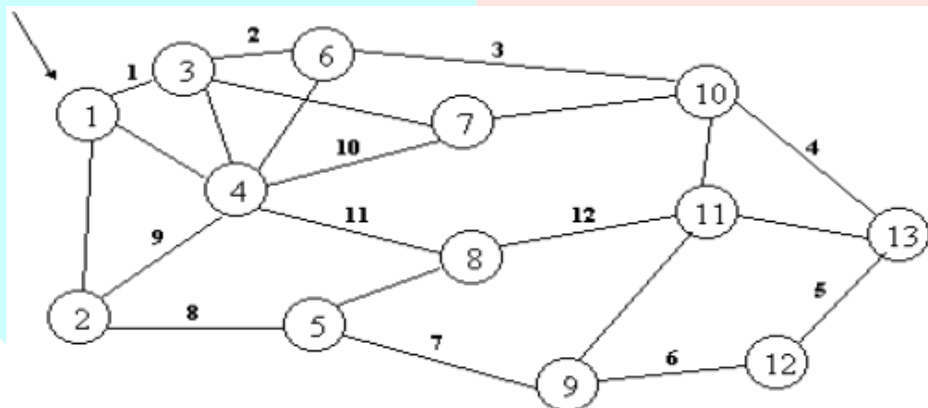
Recorrer un árbol consiste en visitar cada uno de los nodos alcanzables a partir de un nodo dado. Dos tipos de recorrido:

- **Recorrido en ANCHURA (BFS o por niveles)**: primero visita los hermanos y después los hijos. Generaliza el recorrido de los árboles con la misma denominación. Usa una estructura de datos de tipo **cola**. Procedimiento:
 - Visitar el vértice inicial.
 - Visitar los nodos adyacentes.
 - Repetir el proceso con los nodos adyacentes a los nodos del paso anterior.



1, 3, 4, 2, 6, 7, 8, 5, 10, 11, 9, 13, 12

- **Recorrido en PROFUNDIDAD (DFS):** primero visita los **hijos**, antes que los hermanos. Es una generalización del recorrido en Preorden de un árbol. Utiliza una estructura de datos de tipo **pila**. Procedimiento:
 - Visitar el vértice inicial.
 - Si es posible, avanzar a un sucesor aún no visitado.
 - En otro caso, retroceder al vértice visitado anteriormente, e intentar continuar el recorrido desde éste.
 - Una vez visitados todos los descendientes del vértice inicial, si quedan vértices no visitados se inicia un recorrido de otra componente del grafo.



1, 3, 6, 10, 13, 12, 9, 5, 2, 4, 7, 8, 11

2. ALGORITMOS

Un **algoritmo** es una secuencia ordenada y finita de operaciones elementales que devuelve un resultado a partir de los datos de entrada.

1. Eficiencia de un algoritmo

Cuando se nos propone realizar un programa para resolver un problema puede ser interesante plantearnos el diseño de varios algoritmos, y de entre éstos escoger el mejor. Pero rápidamente surge la pregunta: **¿cuál es el mejor?** ¿con qué criterios se decide cuándo un algoritmo es mejor que otro?

Esta cuestión puede tener varias respuestas alternativas:

- Aquel que consuma menos memoria.
- Aquel que finalice la ejecución más velozmente.
- Aquel que funcione adecuadamente (que realice la misión para la cual ha sido diseñado).
- Aquel que sea el más fácil para un humano de leer, escribir o entender.

¿Cómo podemos realizar el estudio de la eficiencia de un algoritmo? Son dos las líneas básicas con las que podemos abordar dicho estudio:

- **Empíricamente:** programar los algoritmos y ejecutarlos varias veces con distintos datos de entrada.
- **Teóricamente:** determinando matemáticamente la cantidad de recursos (tiempo de ejecución y memoria) requeridos por la implementación en función del tamaño de la entrada.

El resultado de un análisis teórico es genérico para cualquier tamaño de la entrada y depende exclusivamente de las instrucciones que componen el algoritmo y del citado tamaño.

La salida de este análisis será una expresión matemática que indique cómo se produce el crecimiento del tiempo que tardaría en ejecutarse el algoritmo conforme aumente el tamaño de la entrada, cuestión que trataremos detalladamente.

El tiempo de ejecución de un algoritmo y su orden de eficiencia

Una vez definido el tamaño de la entrada, resulta conveniente usar una función, **$T(n)$** , para representar el número de unidades de tiempo (segundos, milisegundos...) que un algoritmo tardaría en ejecutarse con unos datos de entrada de tamaño n . Como el tiempo de ejecución de un programa depende claramente del ordenador que se utilice para medirlo y del traductor con el que se haya generado el código objeto, sería preferible que $T(n)$ no represente un tiempo, sino el número de instrucciones simples (asignaciones, comparaciones, operaciones aritméticas...) que se ejecutan, o de forma equivalente, el

tiempo de ejecución del algoritmo en un ordenador idealizado, donde cada una de las instrucciones simples consumen una unidad de tiempo. En general se suele dejar sin especificar las unidades empleadas en $T(n)$ y se asume que $n \geq 0$ y $T(n)$ es positivo.

Un concepto que nos ayudará a entender por qué podemos evitar el uso de unidades de tiempo en la función $T(n)$ es el que se conoce como el **principio de invarianza**, válido independientemente tanto del ordenador que estemos utilizando así como del compilador.

El principio establece que dos implementaciones distintas de un mismo algoritmo, que toman $t_1(n)$ y $t_2(n)$ unidades de tiempo, respectivamente, para resolver un problema de tamaño n , no diferirán en eficiencia en más de una constante multiplicativa. Expresado matemáticamente, existe un $c > 0$ / $t_1(n) \leq ct_2(n)$.

Así, se podrá hacer que un programa vaya 10 o 1.000 veces más rápido cambiando de máquina, pero sólo un cambio de algoritmo nos permitirá obtener una mejora mayor cuanto más crezca el tamaño de la entrada, lo que nos llevará a **ignorar las constantes multiplicativas** a todos los efectos.

Ahora bien, un mismo algoritmo puede ejecutarse con conjuntos de datos diferentes y su tiempo de ejecución puede ser distinto para cada uno de ellos: habrá datos de entrada con los que el algoritmo emplee más tiempo, y otros con los que finalice antes, por lo que es conveniente indicar si la expresión que se obtiene al realizar el análisis de la eficiencia corresponde:

- **Al caso PEOR:** estamos indicando un **límite superior**, el máximo valor, del tiempo de ejecución para cualquier entrada al algoritmo.
- **Al caso PROMEDIO:** será una media ponderada de ambos casos, aunque los abundantes y a veces complicados cálculos para realizar un análisis de eficiencia del caso promedio complican notablemente su uso.
- **Al mejor CASO:** donde ofrecemos un límite inferior, indicando que el algoritmo no se ejecutará por debajo de ese tiempo.

Aunque el **peor caso** suele ser bastante **pesimista**, y probablemente el comportamiento del algoritmo sea algo mejor que el obtenido por el análisis, se suele utilizar más a menudo que el mejor.

¿Cómo se calcula el tiempo de ejecución de un algoritmo? Sobre la base de las instrucciones que componen el algoritmo. Aunque posteriormente lo estudiaremos con detalle, esbozaremos seguidamente algunas ideas.

- La evaluación de una expresión tendrá como tiempo de ejecución lo que se tarde en realizar las operaciones que contenga.
- Una asignación a una variable simple de una expresión, al igual que una operación de escritura de una expresión, suele tardar el tiempo de evaluar dicha expresión más un tiempo constante relativo a gestión interna de la asignación o del proceso de escritura.
- Una lectura de una variable requiere un tiempo constante.

- Una secuencia de instrucciones, la suma de los tiempos de cada una de las instrucciones.
- En una sentencia condicional, su tiempo de ejecución será el de evaluar la condición más el máximo de los costes del bloque IF y del bloque ELSE.
- Para un bucle, se evalúa el tiempo del cuerpo del bucle y se multiplica por el número de iteraciones más el tiempo de evaluar la condición del bucle.

Todas aquellas instrucciones cuyo tiempo de ejecución queda limitado superiormente por una constante, que sólo depende de la implementación, se denominarán **operaciones elementales**.

Por tanto, al habernos desprendido de las constantes multiplicativas, para el análisis de la eficiencia de un algoritmo sólo será relevante el número de operaciones primitivas y no su duración.

Veamos un primer ejemplo para identificar intuitivamente estos conceptos. La siguiente función obtiene la posición donde se encuentra el mínimo valor de un vector de números enteros:

```
/* 1 */ int BuscarMinimo(int *Vector, int n)
/* 2 */ {
/* 3 */   int j, min;
/* 4 */   min= 0;
/* 5 */   for (j=1; j<n;j++)
/* 6 */     if (Vector[j] < Vector[min])
/* 7 */       min= j;
/* 8 */   return min;
/* 9 */ }
```

En el tiempo de ejecución de este programa no se consideran todas las instrucciones: las declaraciones de variables y la propia declaración de la función no intervienen, por lo que empezaremos por la línea 4 a hacer el cálculo del tiempo de ejecución.

- La línea 4 pertenece a una asignación, por lo que contabilizará una constante c_a .
- En la quinta línea se tendrán en cuenta dos constantes: una por el incremento del contador j , c_i , y otra por la evaluación de la condición booleana, c_e , que incrementarán el tiempo de ejecución tantas veces como se ejecute el bucle: $n-1$ veces, más dos veces adicionales correspondientes al caso en que $j == n+1$.
- El cuerpo del bucle podrá tener como coste c_e o $c_e + c_a$, dependiendo de si la condición del if se evalúa verdadera o falsa, y se ejecuta o no el bloque then. Como hemos considerado trabajar siempre con el peor caso, tomaremos la suma del tiempo de evaluación de la condición (línea 6) más el de la asignación de la línea 7.
- Por último, nos queda la línea 8, que consumirá una constante c_r .

Agrupando todo lo anterior en una única expresión tendríamos:

$$T(n) = c_a + (c_i + c_e) (c_e + c_a) (n - 1) + c_r$$

Si consideramos que esas constantes son la unidad, por provenir de operaciones elementales, haciendo cálculos obtendremos $T(n) = 4 (n - 1) + 4 = 4n$.

En este momento ya estamos en condiciones de aclarar el concepto de eficiencia, el cual hace referencia a la forma en que el tiempo de ejecución (y en general cualquier recurso) necesario para procesar una entrada de tamaño n crece cuando se incrementa el valor de n .

Es por esto, por lo que se especificará mediante una función de crecimiento. Como se puede observar, dicha eficiencia sólo depende del tamaño del problema, dejando a un lado cuestiones como la velocidad del ordenador y la eficiencia del compilador.

Se dice que un algoritmo necesita un tiempo de ejecución del orden de una función cualquiera $f(n)$, cuando existe una constante positiva c y una implementación del algoritmo que resuelve cada instancia del problema en un tiempo acotado superiormente por $cf(n)$, siendo la función $f(n)$ la que marca cómo crecerá dicho tiempo de ejecución cuando aumente n . Un algoritmo será, por tanto, más eficiente que otro si el tiempo de ejecución del peor caso tiene un orden de crecimiento menor que el segundo.

Notación asintótica O

La notación O mayúscula, $O[f(n)]$, representa el conjunto de funciones g que crecen como mucho tan rápido como f , o lo que es lo mismo, las funciones g tales que f llega a ser en algún momento una cota superior para g .

En definitiva, se trata de buscar una función sencilla, $f(n)$, que acote superiormente el crecimiento de otra $g(n)$, en cuyo caso se notará como $g(n) \in O[f(n)]$ (g es del orden de f).

La definición formal es la siguiente:

sea $f: \mathbb{N} \rightarrow \mathbb{R} + \cup \{0\}$ una función cualquiera, el conjunto de las funciones del orden de $f(n)$, notado como $O[f(n)]$, se define:

$$O[f(n)] = \{ g \mid \exists c_0 \in \mathbb{R}^+ \text{ y } \exists n_0 \in \mathbb{N}, \forall n \geq n_0 \ g(n) \leq c_0 f(n) \}$$

Esta definición garantiza que si el tiempo de ejecución de una implementación de un algoritmo es $g(n)$, la cual es del orden de $f(n)$, el tiempo $g'(n)$ empleado por cualquier otra implementación que difiera de la primera en el lenguaje y en el compilador utilizado, o la propia máquina, también será del orden de $f(n)$. Es decir, la variación es función de una constante.

Ejemplos:

$T(n) = 3n + 2 \in O(n)$, ya que existen dos constantes positivas $n_0 = 2$, y $c_0 = 4$, tal que $3n + 2 \leq 4n$.

$T(n) = 1.000n^2 + 100n - 6 \in O(n^2)$, porque existen $n_0 = 100$, y $c_0 = 1.000$ que hacen que se cumpla que $1.000n^2 + 100n - 6 \leq 1.001n^2$.

$T(n) = 6 \cdot 2^n + n^2 \in O(2^n)$ debido a que se pueden encontrar dos constantes $n_0 = 4$, y $c_0 = 7$ que hacen que $6 \cdot 2^n + n^2 \leq 7 \cdot 2^n$.

Funciones de complejidad en tiempo más usuales

Las funciones de complejidad algorítmica ordenadas de **mayor a menor eficiencia** son:

- **$O(n) = 1$: constante**, es la complejidad más deseada.
- **$O(n) = \log n$: logarítmica**. Esta complejidad suele aparecer en determinados algoritmos con iteración o recursión no estructural (por ejemplo, búsqueda **binaria**).
- **$O(n) = n$: lineal**. En general esta complejidad es buena y bastante usual. Suele aparecer en la evaluación de un bucle simple cuando la complejidad de las operaciones interiores es constante o en algoritmos con recursión estructural.
- **$O(n) = n \log n$: quasilineal**. También aparece en algoritmos con recursión no estructurada.
- **$O(n) = n^2$: cuadrática**. Aparece en bucles o recursiones **doblemente** anidados.
- **$O(n) = n^3$: cúbica**, aparece en bucles o **recursiones triples**.
- **$O(n) = n^k$: polinómica** ($k > 3$). Si k crece, la complejidad del algoritmo es bastante mala.
- **$O(n) = k^n$: exponencial**. Debe evitarse en la medida de lo posible. Puede aparecer en un subprograma **recursivo** que contenga dos o más llamadas internas. En problemas donde aparece esta complejidad suele hablarse de **explosión combinatoria**.

2. Recursividad

Es una técnica de resolución de problemas muy potente ya que muchos problemas que a primera vista parecen poseer una solución difícil, son resueltos de manera sencilla y elegante. Técnica denominada también **"divide y vencerás"**.

Este método divide el problema original en varios más pequeños que son del mismo tipo que el inicial, procediendo seguidamente a encontrar la solución de estos subproblemas, soluciones en las que se basará la del problema inicialmente planteado. Para ello, realiza de nuevo las correspondientes divisiones en problemas más pequeños aún, hasta que éstos tengan un tamaño tan pequeño para que su solución se conozca de forma directa. Una vez conocidas estas soluciones, podremos ir resolviendo sucesivamente los problemas más grandes hasta llegar a la solución buscada del problema primeramente planteado. Es de vital importancia que existan dichas soluciones a los problemas más sencillos, porque nos permitirán, a partir de ellas, ir resolviendo los problemas más complejos.

Características de los problemas que se resuelven de forma recursiva:

- Los problemas pueden ser **redefinidos en términos de uno o más subproblemas, idénticos** en naturaleza al problema original, pero **menos en tamaño**.
- Uno o más subproblemas tienen **solución directa o conocida**, no recursiva.

- Aplicando la redefinición del problema en términos de problemas más pequeños, dicho problema se reduce sucesivamente a los subproblemas cuyas soluciones se conocen directamente.
- La solución a los problemas más simples se utiliza para construir la solución al problema inicial.

Cuando plasmamos la solución de un problema de manera recursiva en un algoritmo diremos entonces que ha sido resuelto mediante un algoritmo recursivo.

Este algoritmo, o en general cualquier módulo, tendrá que realizar una o varias llamadas a sí mismo.

Es muy importante que siempre se determine en qué momento se detendrán las llamadas recursivas del módulo, ya que si no se hace, se corre el riesgo de no llegar nunca a la solución del problema inicial por estar llamándose el módulo a sí mismo de forma infinita.

Ejemplo: Factorial.

```
long factorial (long n) {  
    if (n == 0) return 1;  
    else return n * factorial(n-1);  
}
```

En esta función podemos claramente determinar dos partes principales:

- La llamada recursiva, que expresa el problema original en términos de otro más pequeño.
- El valor para el cual se conoce una solución no recursiva. Esto es lo que se conoce como caso base: una instancia del problema cuya solución no requiere de llamadas recursivas.

Ejemplo: Secuencia de Fibonacci 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ... Cada término se obtiene sumando los dos términos anteriores.

```
int fibonacci(int n) {  
    if ((n == 1) || (n == 2)) return 1;  
    else return (fibonacci(n-2) + fibonacci(n-1));  
}
```

3. Algoritmos de búsqueda

Se presentan a continuación una serie de algoritmos cuyo objetivo es la localización de un elemento dentro de la estructura de datos. Tenemos:

- **Búsqueda NO informada (exhaustiva):**
 - Búsqueda **secuencial**.
 - Búsqueda **binaria** o dicotómica.
 - Búsqueda por **interpolación**.
 - Búsqueda en **anchura**.
 - Búsqueda en **profundidad**: una variante es el algoritmo **Backtracking** o vuelta atrás. Cada nodo es una solución parcial, si es incorrecta, se vuelve al nodo anterior podando el "árbol". Ejemplos de resolución de problemas: **8 reinas** y **sudokus**.
- **Búsqueda informada (uso de una función heurística):**
 - Algoritmo **Greedy** o devorador (ejemplos: **Kruskal**, **Prim**, **Dijkstra**). Resuelve problemas de **optimización** (obtener el máximo o el mínimo) **no garantizando la obtención de una solución óptima**.
 - Algoritmo de búsqueda **A***.
 - Algoritmo de **escalada**.
 - **Minimax**: utilizado en teoría de juegos y toma de decisiones para encontrar la estrategia óptima **en situaciones de competencia** o conflicto entre **dos jugadores o agentes**.

Búsqueda SECUENCIAL

La forma más simple de búsqueda es la búsqueda secuencial. Esta búsqueda es aplicable a una tabla organizada como un vector.

Supongamos que k es un vector de n claves, de $k(0)$ a $k(n-1)$. Supongamos también que key es un argumento de búsqueda. Queremos obtener el entero i más pequeño tal que $k(i)$ sea igual a key si existe tal i y -1 en caso contrario. El algoritmo para hacerlo es el siguiente:

```
for (i = 0; i < n; i++)  
    if (key == k(i))  
        return(i);  
return(-1);
```

Respecto a la eficiencia de la búsqueda secuencial, el número de comparaciones depende del lugar de la tabla donde aparece el registro que tienen la clave del argumento. Si el registro es el primero de la tabla, se realiza una sola comparación; si el registro es el último, se necesitan n comparaciones. Si es igualmente probable que el argumento aparezca en cualquier posición dada en la tabla, una búsqueda exitosa haría (en el promedio) $n/2$ comparaciones y, una infructuosa n comparaciones. En cualquier caso, su complejidad es **$O(n)$** .

Búsqueda BINARIA

El método de búsqueda más eficiente en una tabla secuencial sin usar índices o tablas auxiliares es el de búsqueda binaria. Básicamente, se compara el argumento con la clave del elemento medio de la tabla. Si son iguales, la búsqueda termina con éxito; en caso contrario se busca de manera similar en la mitad superior o inferior de la tabla.

La mejor manera de definir la búsqueda binaria es la forma recursiva. Sin embargo, la sobrecarga asociada a la recursividad puede hacerla inapropiada para su uso en situaciones prácticas en las cuales la eficiencia se considera primordial. En consecuencia, presentamos la siguiente versión no recursiva del **algoritmo de búsqueda binaria**:

```
low = 0;
hi = n - 1;
while (low <= hi) {
    mid = (low + hi)/2;
    if (key == k(mid))
        return(mid);
    if (key < k(mid))
        hi =mid - 1;
    else
        low = mid + 1;
} /* fin de while */
return (-1);
```

Respecto a la eficiencia de este algoritmo, podemos afirmar que **cada comparación** en la búsqueda binaria **reduce el número de posibles candidatos** en un factor de dos. Así, el número máximo de comparaciones de claves es de manera aproximada $\lg n$. Por tanto, la complejidad del algoritmo de búsqueda binaria es **$O(\lg n)$** .

Búsqueda por INTERPOLACIÓN

Si las claves están **distribuidas de manera uniforme** entre $k(0)$ y $k(n-1)$, el método puede ser aún más eficiente que la búsqueda binaria.

En principio, como en la búsqueda binaria, low se hace 0 y high se hace $n-1$, y a través del algoritmo, se sabe que la clave argumento key está entre $k(\text{low})$ y $k(\text{high})$. Suponiendo que las claves están distribuidas de manera uniforme entre esos dos valores, se esperaría que key estuviese de forma aproximada en la posición:

$$\text{mid} = \text{low} + (\text{high} - \text{low}) * [(key - k(\text{low})) / (k(\text{high}) - k(\text{low}))]$$

Si key es menor que $k(\text{mid})$ haga high igual a $\text{mid}-1$; si es mayor, haga low igual a $\text{mid}+1$. Repetir el proceso hasta que la clave haya sido encontrada o $\text{low} > \text{high}$.

Respecto a la eficiencia de este tipo de búsqueda, si las claves están distribuidas de manera uniforme a lo largo del vector la búsqueda por interpolación requiere un promedio de $\lg(\lg n)$ comparaciones y es raro que requiera muchas más, comparado con la búsqueda binaria que requiere $\lg n$. Sin embargo, si las claves no están distribuidas de manera uniforme, la búsqueda por interpolación puede tener un comportamiento promedio muy pobre.

Ejemplo:

Posición 0 1 2 3 4 5 6 7 8 9

Valor 1 2 3 4 5 6 7 8 9 10

y queremos buscar el elemento que tenga por valor de clave 3, tendríamos lo siguiente:

Valores iniciales de las variables:

low = 0

high = (10 - 1) = 9

key = 3

Calculamos mid con la fórmula anterior y obtenemos:

$mid = 0 + (9 - 0) * [(3 - k(0)) / (k(9) - k(0))]$

Sustituimos los valores de las k, que son: $k(0) = 1$ y $k(9) = 10$, y obtenemos que $mid = 0 + 9 * 2 / 9 = 2$,

Comparamos el valor de $k(2)$ con key (el buscado) y como son iguales acabamos.

Árboles binarios de búsqueda

La propiedad de árbol binario de búsqueda hace que sea muy simple diseñar un procedimiento para realizar la búsqueda. Para determinar si k está presente en el árbol la comparamos con la clave situada en la raíz, r . Si coincide la búsqueda finaliza con éxito, si $k < r$ es evidente que k , de estar presente, será un descendiente del hijo izquierdo de la raíz, y si es mayor será un descendiente de hijo derecho.

Complejidad: $O(\log n)$.

4. Algoritmos de ordenación o clasificación

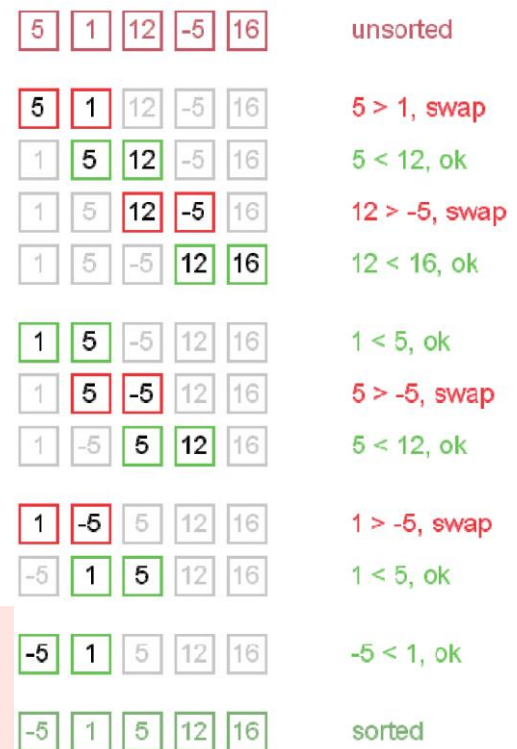
Podemos clasificar los algoritmos de ordenación en:

- **Algoritmos SIMPLES de clasificación:** complejidad $O(n^2)$
 - o Burbuja (bubblesort).
 - o Inserción (insertion sort).
 - o Selección (selection sort).
- **Algoritmos RÁPIDOS de clasificación:** eficiencia superior.
 - o Quicksort.
 - o Mergesort.
 - o Heapsort.
 - o Otros.

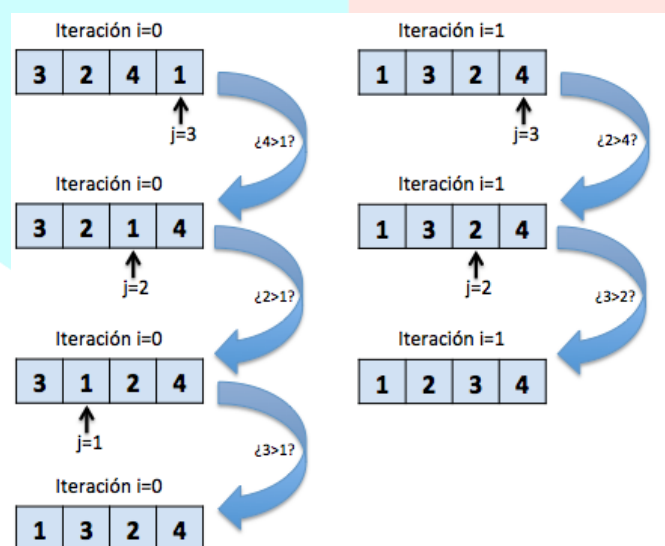
Clasificación por intercambio o BURBUJA (bubblesort)

La idea básica de éste es imaginar que los registros a ordenar están almacenados en un vector vertical. Se recorre varias veces el vector de abajo hacia arriba, y al hacer esto si hay dos elementos adyacentes que no están en orden se invierten. El efecto producido por esta operación es que en el primer recorrido el elemento con menor valor sube al primer elemento del vector. En el segundo recorrido, el siguiente elemento con menor valor, pasa a la segunda posición del vector, y así, sucesivamente. Algoritmo:

```
for (i=0; i < N-1; i++)
    for (j=N-1; j > i; j--)
        if (A[j] < a[j-1]) {
            temp = A[j];
            A[j] = A[j-1];
            A[j-1] = temp;
        }
```



Otro ejemplo:



Variante: burbuja bidireccional (cocktail sort). En cada iteración ordena a los elementos menor y mayor. Menos comparaciones que versión original pero misma complejidad $O(n^2)$.

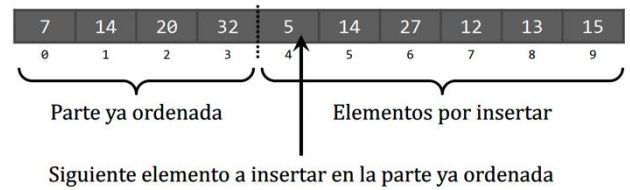
Clasificación por INSERCIÓN (insertion sort)

Denominado así porque en el **i-ésimo recorrido** se inserta el **i-ésimo elemento**, $A[i]$, en el **lugar correcto**, entre $A[0]$, $A[1]$, ..., $A[i-1]$, los cuales fueron ordenados previamente. Después de hacer esta inserción, se encuentran ordenados los elementos $A[0] \dots A[i]$. Algoritmo:

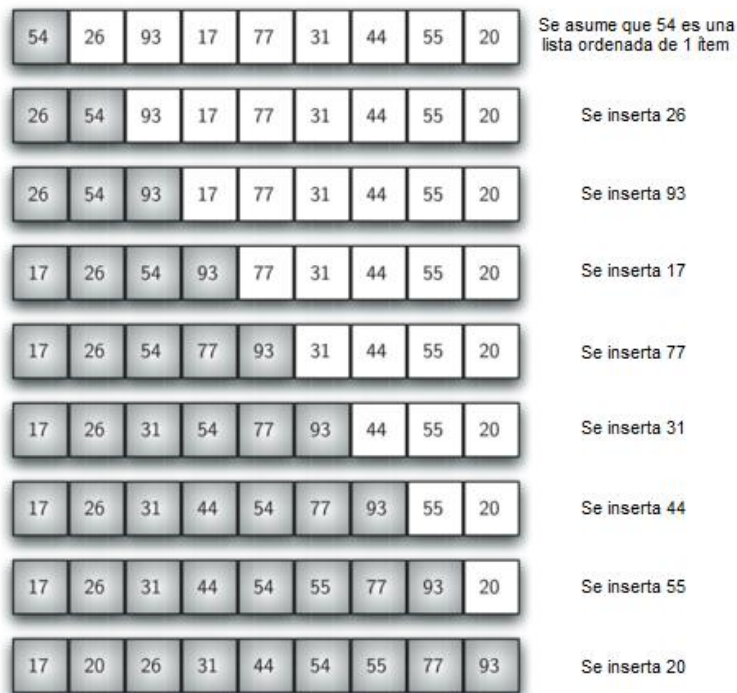
```
for (i=1; i < N-1; i++) {
    temp=A[i];

    for (j=i; (j>0) && (temp<A[j-1]); j--)
        A[j] = A[j-1];

    A[j] = temp;
}
```



Otro ejemplo:



Clasificación por **SELECCIÓN** (selection sort)

La idea es que en el i -ésimo recorrido se selecciona el elemento con el valor menor entre $A[i], \dots, A[N-1]$ y se intercambia con $A[i]$, es decir, se selecciona el siguiente elemento menor de los que quedan. Como resultado, después de i pasadas, los i elementos menores ($A[0] \dots A[i]$) están ordenados. Algoritmo:

```
for (i=0, i < N-1, i++) {
    // Menor elemento vector A[i...N-1]
    pos_min = i;
    temp = A[pos_min];
    for (j=i+1; j < N; j++)
        if (A[j] < temp)
            pos_min = j;

    // Coloca el mínimo en A[i]
    temp = A[i];
    A[i] = A[pos_min];
    A[pos_min] = temp;
}
```

a[0] a[1] a[2] a[3] a[4]

51 21 39 80 36

↑
pasada 1

Pasada 1: Seleccionar 21
Intercambiar 21 y a[0]

21 51 39 80 36

↑
pasada 2

Pasada 2: Seleccionar 36
Intercambiar 36 y a[1]

21 36 39 80 51

↑
pasada 3

Pasada 3: Seleccionar 39
Intercambiar 39 y a[2]

21 36 39 80 51

Pasada 4: Seleccionar 51
Intercambiar 51 y a[3]

Quicksort

Este algoritmo (**inestable**) se basa en el concepto del **pivote**.

Pasos:

- Elegir un elemento de la **lista a ordenar**, al que llamaremos pivote.
- Posicionar el **resto de elementos de la lista**, de manera que todos los elementos menores que él **queden a la izquierda y los mayores que él a la derecha**.
- Repetir este proceso **para cada sublista**.

```
void quicksort (double v[], int izda, int dcha)
{
    int pivote; // Posición del pivote

    if (izda < dcha) {

        pivote = partir (v, izda, dcha);

        quicksort (v, izda, pivote-1);

        quicksort (v, pivote+1, dcha);

    }
}
```

Complejidad: en tiempo **promedio $O(n \log n)$** y en el **peor caso $O(n^2)$** .

Mergesort

Se le considera de ordenamiento externo, dedicado a grandes cantidades de información como el almacenamiento persistente. Está basado en la técnica **"divide y vencerás"**.

Funciona de la siguiente manera:

- Dividir una lista desordenada en dos partes de aproximadamente la mitad del tamaño.
- Ordenar cada lista, recursivamente.
- Mezclar las dos sublistas.

A	L	G	O	R	I	T	H	M	S
---	---	---	---	---	---	---	---	---	---

A	L	G	O	R	I	T	H	M	S
---	---	---	---	---	---	---	---	---	---

A	G	L	O	R	H	I	M	S	T
---	---	---	---	---	---	---	---	---	---

A	G	H	I	L	M	O	R	S	T
---	---	---	---	---	---	---	---	---	---

Complejidad **$O(n \log n)$** .

Heapsort

Un **montículo** es una estructura de datos de tipo **árbol** con información perteneciente a un conjunto ordenado. Puede ser máximo o mínimo, de manera que o bien se sabe que el padre tiene siempre un valor mayor que los hijos o se sabe que el padre tiene un valor menor que los hijos.

Así, el algoritmo consiste en ordenar todos los elementos en un montículo y luego ir extrayéndolos en orden.

Complejidad **$O(n \log n)$** .

Otros algoritmos de ordenación

ALGORITMO	COMPLEJIDAD
Shell sort	$O(n \log^2 n)$
Bucket sort o Binsort	$O(n^2)$ Si elementos distribuidos uniformemente $O(n)$ Uso del concepto de urna/casillero: contiene elementos que cumplen un conjunto de condiciones
Counting sort	$O(n+k)$
Radix sort	$O(nk)$
Cubesort	$O(n \log n)$

5. Algoritmos teoría de grafos

Un **árbol de expansión mínimo** o **árbol recubridor mínimo** o **árbol generador mínimo** es, dado un grafo conexo, ponderado y no dirigido, un subgrafo formado por un árbol que contiene todos los vértices del grafo inicial tal que el coste es mínimo. A partir de esta definición de árbol recubridor mínimo tenemos dos algoritmos:

- El algoritmo de **Kruskal** es un algoritmo de la teoría de grafos para encontrar un árbol recubridor mínimo en un grafo conexo, no dirigido y ponderado. Empezando por cualquier arista, construye incrementalmente el árbol, seleccionando en cada paso una arista.
- El algoritmo de **Prim** es un algoritmo perteneciente a la teoría de los grafos para encontrar un árbol recubridor mínimo en un grafo conexo, no dirigido y cuyas aristas están etiquetadas. Empezando por cualquier vértice, construye incrementalmente el árbol, seleccionando en cada paso una arista.

En relación con la **búsqueda de caminos mínimos** citamos los siguientes algoritmos:

- El algoritmo de **Dijkstra**, también denominado algoritmo de **caminos mínimos** es un algoritmo para la determinación del camino más corto dado un vértice origen al **resto de vértices en un grafo con pesos en cada arista**.
- El algoritmo de **Bellman-Ford** es como el algoritmo de Dijkstra, pero en el que el peso de las aristas puede ser **negativo**.
- El algoritmo de **búsqueda A*** busca el camino más corto **entre dos vértices** guiado por una función heurística.
- El algoritmo de **Floyd-Warshall** encuentra el camino más **corto entre todos los pares** de nodos de un grafo.

3. ORGANIZACIONES DE FICHEROS

Un fichero es un conjunto de registros, organizados según unas reglas lógicas y almacenados en determinados soportes físicos. Este almacenamiento se realizará atendiendo unas determinadas reglas, que es lo que determina la organización del fichero.

Los **registros** unidades homogéneas de información dentro del fichero que contienen la información organizada en campos.

Un **campo** es la unidad mínima de procesamiento con significado propio. Por su relevancia existe el campo denominado **clave** o identificador, el cual que permite identificar unívocamente un registro del resto.

Básicamente los dos modos de organizar los ficheros son:

- **Organización secuencial**: se caracteriza fundamentalmente por almacenar los registros en posiciones físicas contiguas de memoria. Variantes:
 - Organización secuencial **indexada**.
 - Organización secuencial **encadenada**.
 - Organización secuencial **indexada encadenada**.
- **Organización directa (aleatoria)**: el almacenamiento físico de los registros se hace a través de un identificativo o clave que indicará la posición del registro en el fichero, el valor de la posición de almacenamiento en el fichero puede obtenerse directamente del valor del identificativo o después de haber aplicado un algoritmo de transformación sobre éste. En este segundo caso, la organización también es conocida como dispersa (hash).
- **Organización en montículos** (ORGANIZACIÓN ESPECIAL): se puede insertar registros en cualquier posición del fichero en el que haya espacio suficiente, no existiendo ninguna ordenación de los registros. Usado en bases de datos, un archivo por tabla.

1. Organización secuencial

El orden de los registros, como el nombre de la organización indica, será secuencial, es decir, uno detrás de otro.

En este caso el campo identificativo, sólo indica el campo clave por el accedemos al registro, para consultarlo, pero no indica el orden del registro dentro del fichero. Este tipo de organización generará los registros en el mismo orden en que se grabaron y el acceso a ellos siempre será, siguiendo esa misma secuencia de grabación.

FICHERO

1	A	0
25	B	1
139	M	2
89	D	3
123	E	4
123	E	5
198	L	6
204	H	7
1	A	8

Ventajas:

- Suponiendo que queremos consultar una gran parte del fichero, o la mayoría de sus registros la organización secuencial presenta una ventaja y es la rapidez con la que se puede acceder a registros colocados en posiciones físicamente contiguas.
- En lo referente a la ocupación del soporte físico, este tipo de organización es la más óptima, ya que no deja huecos pues los registros se graban unos a continuación de otros

Desventajas:

- Es muy lento para consultar solamente unos pocos registros.
- Cuando surge la necesidad de insertar nuevos registros, o de modificar cualquiera ya existente. Esto implica que en cualquiera de los dos casos el fichero tendrá que ser creado de nuevo. Cuando se utiliza un fichero secuencial, este sólo podrá ser utilizado para leer o para escribir, pero nunca para leer y escribir simultáneamente.

2. Organización directa

En este tipo de organización, el concepto de identificativo o clave tiene sentido, ya que será éste el que indique la posición (DIRECCIÓN) física de almacenamiento.

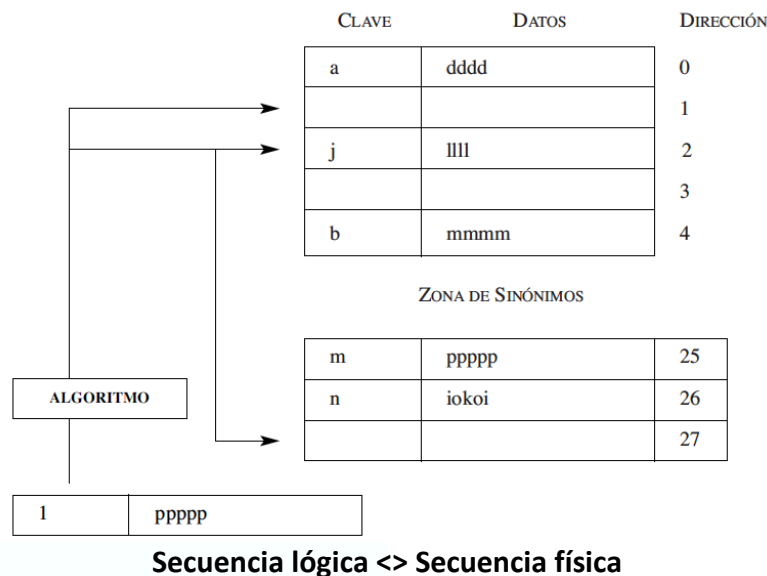
Si la dirección física de almacenamiento es obtenida directamente del contenido del identificativo (**direccionamiento directo**), fácilmente podemos deducir que la secuencia lógica de almacenamiento de los registros y la secuencia física coinciden:

	CLAVE	DATOS	DIRECCIÓN
	0	dddd	0
			1
	2	llll	2
			3
	4	mmmm	4

1	ppppp
---	-------

Secuencia lógica = Secuencia física

Si la dirección de almacenamiento se obtiene del identificativo, pero después de una transformación, a través de un algoritmo (**direccionamiento indirecto**), entonces la secuencia lógica de almacenamiento de los registros y la secuencia física no coinciden:



En general, un fichero con organización directa tiene las siguientes ventajas:

- Acceso inmediato a los registros.
- No es necesario ordenar el fichero.
- No es necesario regenerar el fichero.
- Rápidos para el tratamiento de pocos registros.

Como inconvenientes tenemos:

- Tratamiento de pocos registros, no recomendado para consultas masivas de registros.
- Genera muchos huecos dentro del fichero. Estos huecos surgen ya que los identificativos de los registros a almacenar, después de haber sufrido la aleatorización, indicarán posiciones de almacenamiento que lo más normal es que no sean contiguas. Esto implica un desaprovechamiento del soporte de almacenamiento respecto del número real de registros almacenados frente al número total de posiciones ocupadas.

3. Variantes de la organización secuencial

Debido a la limitación de este tipo de organización, surgen otros tipos de organización, en las que será necesario manejar 2 conceptos, que se utilizarán exclusivamente en ficheros con organización secuencial y sirven para agilizar procesos de consulta:

- **Puntero**: posición de memoria que contiene la dirección de una información dentro del fichero.

- **Índice:** tabla cuyos elementos contienen direcciones de datos a los que se puede acceder de forma secuencial.

Organización SECUENCIAL INDEXADA

En este modo de organización, al fichero le acompaña un fichero de índice que tiene la función de permitir el acceso directo a los registros del fichero de datos.

TABLA DE ÍNDICES



68	0
138	3
205	6

FICHERO

CLAVES	DATOS	DIRECCIONES
12	A	0
25	B	1
68	C	2
89	D	3
123	E	4
138	F	5
198	G	6
204	H	7
205	I	8

Características:

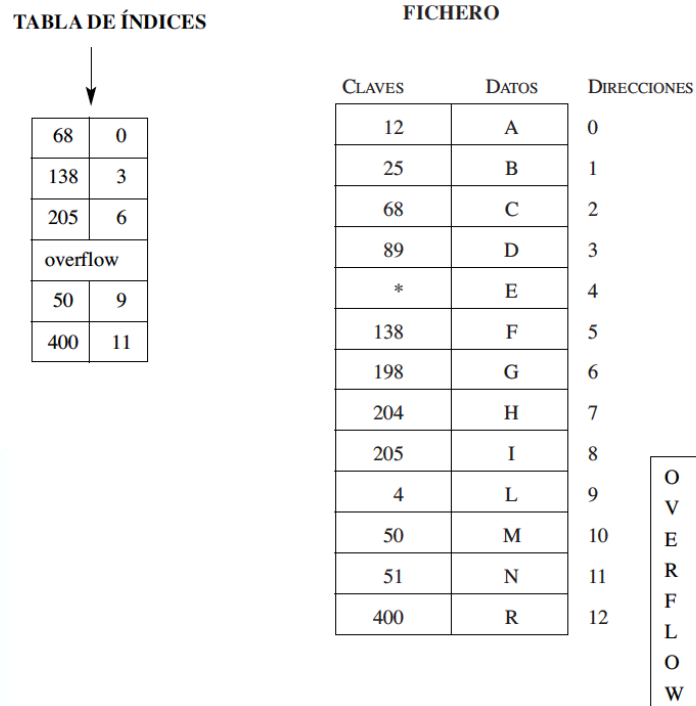
- Aprovecha lo mejor de organización secuencial: manejo grandes volúmenes de información.
- Aprovecha lo mejor de organización directa: acceso rápido y directo a los registros.
- Eficaz tanto para consultas esporádicas como consultas completas.

Existen tres partes en la organización de la información:

- En la que se almacenan los datos ordenados.
- Se almacenan las tablas de índices secuenciadas y ordenadas.
- Almacenamiento de desborde (u overflow) para nuevas inserciones.

En la organización secuencial indexada, se crea una la zona de desbordamiento para almacenar los registros nuevos insertados. Obviamente la secuencia lógica de fichero en esta organización se romperá.

De cualquier manera, utilizar zona de excedentes depende directamente de las necesidades del usuario. No hay que olvidar que los registros en esta zona se almacenarán en orden de llegada, por lo que esta zona siempre que contenga registros, lo más probable es que no estén ordenados.



Una vez creado el fichero, para introducir nuevos registros hay que añadir una zona de datos de excedentes o de overflow. Se rompe la secuencia lógica del fichero y la ordenación inicial. Caso de eliminación, esta no se efectuará de forma real, sino que se marca el registro (*) en el gráfico) para no perder la secuencia lógica. En un proceso posterior se podrían eliminar.

Organización SECUENCIAL ENCADENADA

Partimos de que, al igual que en los ficheros secuencial indexados, los ficheros secuenciales encadenados tienen la misma base que los secuenciales puros, pero sirviéndose de punteros con un fin muy concreto.

El puntero es un campo adicional que se incluye en el registro que indica cual es el siguiente o anterior registro en secuencia lógica.

Características:

- Las inserciones se hacen al final. Se encadenan lógicamente utilizando los punteros. Secuencia física y lógica no coinciden.
- El borrado consiste en marcar el registro pero manteniendo el puntero para conservar la estructura lógica de encadenamiento.
- Para localizar el primer registro se utiliza el campo cabecera que se comporta como un puntero para el comienzo de la cadena.

CLAVES	DATOS	PUNTERO	DIRECCIONES
		4	0
2	B	7	1
68	C	3	2
89	D	5	3
1	E	1	4
138	F	6	5
198	G	8	6
3	H	2	7
205	I	0	8

Organización SECUENCIAL INDEXADA ENCADENADA

Como alternativa a estos dos tipos de organización (indexada y encadenada), surge la organización secuencial indexada encadenada aprovechando lo mejor de sus compañeras más directas. Destaca fundamentalmente por utilizar **punteros e índices** simultáneamente.

Claramente se deduce que si usamos punteros es para que la secuencia lógica no se pierda. Sabemos que si se utilizan índices, las inserciones van a parar a la zona de desbordamiento rompiendo la secuencia lógica. Es entonces cuando el puntero realiza la función de encadenar la zona real con la zona de desbordamiento. Este encadenamiento implica que la secuencia lógica no se rompa. En este tipo de ficheros, los índices tienen el mismo funcionamiento que en la secuencial indexada, es decir, permitir acceso casi directo a registros situados en la zona real del fichero.

TABLA DE ÍNDICES

68	0
138	3
205	6
overflow	
50	9
400	11

CLAVES	DATOS	PUNTERO	DIRECCIONES
		9	0
25	B	10	1
68	C	3	2
89	D	5	3
*	E	5	4
138	F	6	5
198	G	7	6
204	H	8	7
205	I	12	8
4	L	21	9
50	M	11	10
51	N	2	11
400	R	0	12

O
V
E
R
F
L
O
W

Los ficheros secuenciales indexados encadenados, necesitan reorganizarse, que consiste en eliminar el área de desbordamiento, eliminar físicamente las bajas lógicas y dejar el fichero ordenado por el campo clave.

TABLA DE ÍNDICES

		CLAVES	DATOS	PUNTERO	DIRECCIONES
25	0			1	0
68	3	4	L	2	1
198	6	25	B	3	2
400	9	50	M	4	3
		51	N	5	4
		68	C	6	5
		89	D	7	6
		138	F	8	7
		198	G	9	8
		204	H	10	9
		205	I	11	10
		400	R	0	11
					12

4. FORMATOS DE INFORMACIÓN Y FICHEROS

1. Formatos de imágenes

Métodos de codificación de compresión **CON pérdida** (la imagen o secuencia reconstruida es más o menos diferente de la imagen original):

- Codificación por **transformación**.
- Vector de **cuantificación**.
- Compresión **fractal**.

Formatos con pérdida: **JPG, TIFF**.

Métodos de codificación de compresión **SIN pérdida** (la imagen reconstruida es exactamente igual que la imagen original):

- **RLE** (Run-Length Encoding).
- Codificación de **Huffman**.
- Codificación **aritmética**.
- Lempel-Ziv (**LZ**) y Lempel-Ziv-Welch (**LZW**).

Formatos sin pérdida: **GIF, PNG, BMP, TIFF, RAW**.

Los formatos de imagen más destacados son:

- **TIFF (Tagged Image File Format)**: es, en principio, un formato muy flexible con o sin pérdida. Los detalles del algoritmo de almacenamiento de la imagen se incluyen como parte del fichero. En la práctica, TIFF se usa casi exclusivamente como formato de almacenamiento de imágenes sin pérdidas y sin ninguna compresión. Consecuentemente, los archivos en este formato suelen ser muy grandes. Algunas veces se usan un algoritmo de compresión sin pérdidas LZW.
- **PNG (Portable Network Graphics)**: es un formato de almacenamiento sin pérdida. Al contrario que ocurre con el formato TIFF puede comprimir la imagen. Además, tal compresión es totalmente reversible y por tanto la imagen que se recupera es exacta a la original. Ofrece profundidades desde 8 bits con paleta optimizada, 24 y 48 bits: 281 billones de colores y en escala de grises se puede obtener archivos desde 8 y 16 bits igual a 65536 tonos de grises.
- **GIF (Graphics Interchange Format)**: crea una tabla de 256 colores a partir de una de 16 millones. Si la imagen tiene menos de 256 colores, GIF puede almacenar la imagen sin pérdidas. Cuando la imagen contiene muchos colores, el software que crea el archivo GIF usa algún algoritmo para aproximar los colores de la imagen con una paleta limitada de 256 colores disponibles. Un buen algoritmo de este tipo tratará de encontrar un conjunto óptimo de 256 colores. Algunas veces, GIF usa el color más cercano para

representar cada píxel, y algunas veces usa un "error de difusión" para ajustar los colores de los píxeles vecinos y así corregir el error producido en cada píxel.

GIF produce compresión de dos formas. Primero, reduce el número de colores de la imagen a 256 y, por tanto, reduce el número de bits necesario por píxel. Después, reemplaza áreas de color uniforme usando código de secuencias: en lugar de almacenar "blanco, blanco, blanco, blanco, blanco" almacena "5 blanco".

Por tanto, GIF es una compresión de imágenes sin pérdida sólo para imágenes de 256 colores o menos. Sin embargo, para una imagen de 16 millones de colores GIF puede "perder" el 99,998% de los colores. Además, permite transparencia y soporta animaciones.

- **JPG (Joint Photographic Experts Group):** imagen en formato comprimido con pérdidas. Ocupa menos que el formato TIFF. Es el método de compresión más adecuado para fotografías e imágenes de tonos continuos similares que contiene muchos colores. Permite obtener unas ratios de compresión muy altos manteniendo a su vez una calidad en la imagen muy elevada. JPG analiza las imágenes y elimina la información que no es apreciable. JPG almacena imágenes de 16 millones de colores. Otro aspecto importante es que el método JPG permite distintos niveles de compresión. En niveles de compresión de imágenes moderado, es muy difícil discernir las diferencias de la imagen original. Programas de tratamiento de imágenes avanzados como GIMP, Paint Shop Pro o Photoshop permiten ver la calidad de la imagen y el tamaño del fichero como una función de nivel de compresión, de esa forma, se puede elegir convenientemente la calidad y el tamaño del fichero deseado.
- **RAW:** es la imagen de salida que ofrece algunas cámaras digitales (en crudo). Aunque es un método sin pérdida, ofrece un factor de tres o cuatro menor que el formato TIFF de la misma imagen. La desventaja es que el método RAW no está estandarizado y cada marca tiene su propia versión de dicho método, por tanto, se debe usar el software de la cámara para poder visualizar las imágenes.
- **BMP (Windows Bitmap):** es un formato de almacenamiento **sin compresión** de imágenes propiedad de **Microsoft**.
- **PSP, PSD y xcf:** son formatos propios de distintos programas de edición: Paint Shop Pro, Photoshop y GIMP, respectivamente.

En resumen, los métodos de compresión TIFF, PNG, GIF y JPG son públicos y, por tanto, se pueden implementar en cualquier programa gráfico. GIF y JPG son los más usados en las páginas web. Dado que PNG hace lo mismo que GIF e incluso mejor, se espera que PNG sustituirá GIF en el futuro. PNG no sustituirá JPG dado que JPG consigue una mayor compresión en imágenes fotográficas.

Resumen extensiones de imágenes

EXTENSIÓN	DESCRIPCIÓN FORMATO
.bmp	Windows Bitmap. Formato comprimido sin pérdidas
.gif	Graphics Interchange Format. Formato comprimido sin pérdidas
.jpeg .jpg	Joint Photographic Experts Group. Formato comprimido con pérdidas
.jp2	JPEG 2000. Intento de sustituir el formato JPEG
.png	Portable Network Graphics. Formato comprimido sin pérdidas
.tif .tiff	Imagen de alta calidad, con o sin pérdidas
.psp	Paint Shop Pro
.psd	Photoshop
.xcf	GIMP
.ai	Adobe illustrator
.cdr	Corel Draw
.dwg	AutoCAD Drawing Database Format 1
.svg	Scalable Vector Graphics
.raw	Imagen RAW

2. Formatos de documentos

Inicialmente, se van a comentar los formatos de documentos más utilizados. En el tema B1T10 se verán los formatos admitidos y que deben ser usados según indica la NTI Catálogo de estándares.

Extensiones de documentos

EXTENSIÓN	DESCRIPCIÓN FORMATO	APP ASOCIADA
.odt .doc .docx	Archivos que contienen textos con formato	Open Office Writer permite abrir los tres tipos de extensiones. Microsoft Office solamente permite abrir la extensión *.doc y *.docx.
.odp .ppt .pptx	Presentación en diapositivas	Open Office Impress permite abrir los dos tipos de extensiones. Microsoft Power Point solamente permite abrir la extensión *.ppt y *.pptx.
.ods .xls .xlsx	Libros u hojas de cálculo	Open Office Calc permite abrir los dos tipos de extensiones. Microsoft Excel solamente permite abrir la extensión *.xls y *.xlsx.
.pdf	Portable Document Format	<u>PDF/A</u> (Archive): archivado. ISO 19005. <u>PDF/E</u> (Engineering): ingeniería. ISO 24517. <u>PDF/H</u> (Healthcare): industria médica. <u>PDF/UA</u> (Universal Accessibility): accesibilidad. ISO 14289. <u>PDF/VCR</u> (Variable Content Replacement): impresión de datos variables. ISO 16613 <u>PDF/VT</u> (Variable & Transactional): impresión variable y transaccional. ISO 16612. <u>PDF/X</u> (eXchange): impresión. ISO 15930.

3. Formatos de intercambio de datos

De entre los formatos de intercambio de datos, actualmente destacan JSON y BSON.

- **JSON (JavaScript Object Notation)**: es un formato ligero de intercambio de datos (estándar ECMA-404), fácil de entender, y ofrece sencillez a las máquinas en la generación e interpretación. Basado en un subconjunto del lenguaje de programación JavaScript (ECMA-262), es completamente independiente del lenguaje, pero utiliza convenciones que son ampliamente conocidos por los programadores de la familia de lenguajes C, incluyendo C, C++, C#, Java, JavaScript, Perl, Python, y muchos otros.

Está construido sobre dos estructuras:

- Una **colección de pares** de nombre/valor. En varios lenguajes, esto se realiza como un objeto, registro, estructura, diccionario, tabla hash, lista con clave o matriz asociativa.
- Una **lista ordenada de valores**. En la mayoría de los lenguajes, esto se realiza como una matriz, vector, lista o secuencia.

Un **valor JSON** puede ser un:

- **Objeto**: es un conjunto **desordenado** de pares **nombre/valor**. Un objeto comienza con llave de apertura ({) y termine con llave de cierre (}). Cada nombre es seguido por dos puntos (:) y los pares nombre/valor están **separados por coma**.

```
{ "name": "John", "age": 30, "car": null }
```

- **Array**: es una colección de valores. Un array comienza con corchete izquierdo ([) y termina con corchete derecho (]). Los valores se separan por coma.

```
{ "coches": [ "Ford", "BMW", "Renault" ] }
```

- **Cadena de caracteres**: es una colección de cero o más caracteres Unicode, encerrados entre comillas dobles, usando barras divisorias invertidas como escape. Una cadena de caracteres es parecida a una cadena de caracteres C o Java.

```
{ "name": "Juan" }
```

- **Número**: es similar a un número C o Java, excepto que no se usan los formatos octales y hexadecimales.

```
{ "age": 30 }
```

```
{  
  "number_1" : 210,  
  "number_2" : 215,  
  "number_3" : 21.05,  
  "number_4" : 10.05  
}
```

- **Valor**: puede ser una cadena de caracteres con comillas dobles, o un número, o true o false o null, o un objeto o un array. Estas estructuras pueden anidarse.

```
{ "Permitido": false } { "vip": null }
```

Ejemplos:

```
{  
  "nombre": "Pepe",  
  "apellidos": "Medina",  
  "edad": 41  
}
```

```
{  
  "squadName": "Super hero squad",  
  "homeTown": "Metro City",  
  "formed": 2016,
```

```
"secretBase": "Super tower",
"active": true,
"members": [
  {
    "name": "Molecule Man",
    "age": 29,
    "secretIdentity": "Dan Jukes",
    "powers": [
      "Radiation resistance",
      "Turning tiny",
      "Radiation blast"
    ]
  },
  {
    "name": "Madame Uppercut",
    "age": 39,
    "secretIdentity": "Jane Wilson",
    "powers": [
      "Million tonne punch",
      "Damage resistance",
      "Superhuman reflexes"
    ]
  },
  {
    "name": "Eternal Flame",
    "age": 1000000,
    "secretIdentity": "Unknown",
    "powers": [
      "Immortality",
      "Heat Immunity",
      "Inferno",
      "Teleportation",
      "Interdimensional travel"
    ]
  }
]
```

- **BSON (Binary JSON)**: es una **serialización** codificada en binario de documentos similares a JSON. Como JSON, BSON admite la incrustación de documentos y matrices dentro de otros documentos y matrices.

BSON representa un superconjunto de JSON, ya que:

- almacena **datos en binario**.
- incluye un conjunto de tipos de datos no incluidos en JSON, como pueden ser **ObjectId, Date o BinData**.

Características:

- **Ligero**: mantener la sobrecarga espacial al mínimo es importante para cualquier formato de representación de datos, especialmente cuando se usa en la red.
- **Transitable**: BSON está diseñado para atravesarlo fácilmente. Esta es una propiedad vital en su función de representación de datos principal para MongoDB.

- **Eficiente:** la codificación de datos a BSON y la decodificación de BSON se pueden realizar muy rápidamente en la mayoría de los lenguajes debido al uso de tipos de datos C.

Resumen formatos de intercambio de datos

EXTENSIÓN	NOMBRE	DESCRIPCIÓN
CSV	Valores separados por coma	Tipo de documentos en formato abierto sencillo para representar datos en formato de tabla, en columnas separadas por comas (o punto y coma, donde la coma es el separador decimal) y las filas son saltos de línea. Los campos que tienen una coma, un salto de línea o una comilla doble deben cerrarse entre comillas dobles. No indica un juego concreto de caracteres, ni cómo van situados los bytes, ni el formato para el salto de línea. Las extensiones que se utilizan son .csv y .txt.
XML	Lenguaje de Etiquetado Extensible	Es un metalenguaje simple pero estricto, desarrollado por W3C . Desarrolla un papel fundamental en el intercambio de una gran variedad de datos. XML es un formato que permite la interpretación de datos a través de varias aplicaciones. Es una simplificación y adaptación del SGML y permite definir la gramática de lenguajes específicos. En realidad, XML es una manera de definir lenguajes para diferentes necesidades.
JSON	Notación de Objetos de JavaScript	Formato ligero de intercambio de datos, fácil de entender, y ofrece sencillez a las máquinas en la generación e interpretación. Basado en un subconjunto del lenguaje de programación JavaScript (ECMA-262), es completamente independiente del lenguaje.
BSON	JSON binario	Es un superconjunto de JSON, una serialización codificada en binario. Utilizado por MongoDB.
YAML	YAML Ain't Markup Language	Formato para el intercambio de información que tiene como objetivo facilitar el mapeo de estructuras de datos más complejas (como listas y arreglos asociativos) en un documento de texto plano legible para un ser humano.

4. Formatos de audio y video

Un **códec** (codificador y decodificador) es una **especificación** sobre cómo codificar un tipo de información (audio, video) para luego poder decodificarla.

Un **contenedor** es un **formato** de archivo, es decir, es una especificación sobre cómo se **almacenan** dentro de un archivo diferentes tipos de contenido multimedia codificado.

Códecs de VIDEO:

- **AV1** (AOM Video 1): **NUEVO** códec de código abierto desarrollado por la Alliance for Open Media (AOMedia).
- **dirac-research**: extensión drc.
- **H.264** y **H.265**: estándares del **ITU-T**.
- **libtheora**: extensión ogg (Fundación Xiph.Org).
- **libvpx** (VP9): de Google.
- **OpenH264** (Cisco).

Formatos de VIDEO:

- **AVI** (Audio Video Interleave): de Microsoft.
- **FLV** (Flash Video): de Adobe.
- **MKV** (Matroska).
- **MOV** (Apple).
- **MPEG-4** (Moving Picture Experts Group): abreviado como MP4, utiliza los códecs H.264 y H.265.
- **Ogg** (Fundación Xiph.Org).
- **WebM** (Google): formato abierto para video y audio. Soporta VP9 y AV1 para video y Opus para audio.
- **WMV** (Microsoft).

Códecs de AUDIO:

- **AAC** (Advanced Audio Coding).
- **FLAC** (Free Lossless Audio Codec).
- **Opus**.
- **Vorbis**.

Formatos de AUDIO:

- AAC.
- AC3.
- G.711, G.722, G.723, G.726, G.728, G.729: de ITU-T.
- FLAC.
- MPEG-1 Audio Layer 2 (MP2).
- MPEG-1 Audio Layer 3 (MP3).
- Vorbis.
- WMA (Windows Media Audio): de Microsoft.

